

مقدمه و تبیین ساختار دوره مدل‌های زبانی بزرگ (LLM)

۱. رویکرد و متدولوژی آموزشی

این دوره آموزشی با تمرکز بر یادگیری عملیاتی (Operational Learning) و پیاده‌سازی پروژه‌های واقعی طراحی شده است. ساختار کلاس بر اساس اصول زیر استوار است:

- مدت زمان دوره: ۴ هفته فشرده.
- رویکرد محتوایی: حجم بالای انتقال مطالب تخصصی. سرعت تدریس به صورت پیش‌فرض بالا در نظر گرفته شده است تا پوشش مطالب حداکثری باشد، اما این سرعت بر اساس بازخورد دانشجویان (Feedback-driven) قابل تنظیم است.
- شیوه تدریس: تعاملی و مبتنی بر نیازسنجی لحظه‌ای. دانشجویان موظف هستند در صورتی که مبحثی تکراری است یا نیاز به توضیح بیشتر دارد، مدرس را مطلع کنند.
- تمرکز عملی: هدف نهایی، خروج از مباحث صرفاً تئوریک و ورود به پیاده‌سازی‌های فنی روی لپ‌تاپ و سرورهای شخصی است.

۲. تحلیل سطح فنی و حوزه‌های کاربردی شرکت‌کنندگان

بر اساس معارفه انجام شده، شرکت‌کنندگان در این دوره در حال کار بر روی پروژه‌های متنوعی در حوزه هوش مصنوعی (Artificial Intelligence) هستند. این تنوع نشان‌دهنده گستردگی کاربرد مدل‌های زبانی و یادگیری عمیق در صنایع مختلف است.

حوزه‌های فعال پژوهشی و عملیاتی کلاس:

- پردازش تصویر و پزشکی: استفاده از شبکه‌های عصبی کانولوشنال (CNN) برای پیش‌بینی بیماری‌هایی نظیر آلزایمر.
- پردازش زبان طبیعی (NLP) و چت‌بات‌ها: توسعه چت‌بات‌های تخصصی (مانند چت‌بات حقوقی) برای سازمان‌های بزرگ دولتی و شهرداری‌ها.
- بازارهای مالی: استفاده از الگوریتم‌های هوش مصنوعی در مدیریت مالی و تحلیل رفتار مصرف‌کننده.
- امنیت سایبری (Cyber Security): تشخیص دیپ‌فیک (Deep Fake) و چالش‌های امنیتی مدل‌های تولیدی.
- تولید گفتار (Speech Synthesis): کار بر روی مدل‌های تبدیل متن به گفتار (Text-to-Speech) و مدل‌های پیشرفته صوتی (مانند اشاره به مدل‌هایی نظیر EFTS یا مشابه آن).

Case Study: کاربرد هوش مصنوعی در مدیریت شهری

صورت مسئله: نیاز به پاسخگویی خودکار به سوالات حقوقی و شهرسازی مراجعین.

راه‌حل پیشنهادی: توسعه چت‌بات‌های هوشمند (LLM-based Chatbots) برای معاونت شهرسازی و شهرداری تهران.

۳. پیش‌نیازها و انتظارات

برای بهره‌برداری حداکثری از این دوره، تسلط نسبی بر موارد زیر توسط دانشجویان توصیه شده است:

۱. آشنایی با مفاهیم پایه یادگیری ماشین. (Machine Learning)
۲. تجربه کار با داده‌کاوی (Data Mining) و زبان‌های برنامه‌نویسی مرتبط (مانند پایتون).
۳. آمادگی برای کار با محیط‌های توسعه عملیاتی. (Hands-on)

خلاصه این بخش

- دوره رویکردی کاملاً عملیاتی و فشرده دارد و هدف آن پیاده‌سازی مدل‌هاست.
- کاربردهای مورد بحث در کلاس شامل حوزه‌های متنوعی از جمله پزشکی، مالی، حقوقی و امنیت سایبری است.
- تعامل دوطرفه و ارائه بازخورد مداوم از سوی دانشجویان برای تنظیم سرعت کلاس الزامی است.
- تمرکز اصلی بر روی تکنولوژی‌های روز مانند Deep Fake Detection، TTS و LLMها خواهد بود.

۴. بررسی تکنولوژی (Tech Stack) مورد استفاده در کلاس

بر اساس تجربیات شرکت‌کنندگان، ابزارها و مدل‌های متنوعی در پروژه‌های جاری کلاس مورد استفاده قرار می‌گیرند. شناخت این ابزارها برای ایجاد زبان مشترک در طول دوره ضروری است.

الف) مدل‌های زبانی و فریم‌ورک‌های توسعه چت‌بات

- مدل‌های متن‌باز: (Open Source Models) استفاده از مدل‌های خانواده Llama (متا) و Gemma (گوگل) برای پیاده‌سازی‌های محلی.
- فریم‌ورک: Rasa استفاده از ابزار Rasa برای توسعه چت‌بات‌های مکالمه‌محور (Conversational AI) که قابلیت شخصی‌سازی بالایی دارد.

ب) مدل‌های صوتی و تبدیل متن به گفتار (TTS)

یکی از مباحث تخصصی مطرح شده، حوزه Text-to-Speech (TTS) است. مدل‌های پیشرفته زیر مورد بحث قرار گرفتند:

- مدل VITS۲ اشاره به مدل‌های مبتنی بر Variational Inference (استنتاج تغییراتی) که کیفیت صدای بسیار طبیعی تولید می‌کنند.
- مدل‌های EfficientTTS: تمرکز بر مدل‌هایی که علاوه بر کیفیت، سرعت پردازش (Inference Time) پایینی دارند.

۵. کاربردهای بین‌رشته‌ای (Interdisciplinary Applications)

یکی از نقاط قوت این دوره، حضور متخصصین از حوزه‌های غیرمهندسی است که نشان‌دهنده نفوذ هوش مصنوعی در تمامی علوم است.

دسته‌بندی پروژه‌های تعریف شده:

۱. بهداشت و درمان (Healthcare):

- تحلیل سیگنال‌های نوار قلب (ECG) با استفاده از هوش مصنوعی.
- انفورماتیک پزشکی و سیستم‌های تشخیص بیماری.

۲. علوم انسانی و فلسفه:

- استفاده از LLM ها در تحلیل متون فلسفه اسلامی. این حوزه یکی از چالش‌برانگیزترین بخش‌هاست، زیرا مدل باید مفاهیم انتزاعی و پیچیده فلسفی را درک و پردازش کند.

۳. مخابرات و تحول دیجیتال:

- پروژه‌های خدمات مشتریان هوشمند در اپراتورهای بزرگ (مانند ایرانسل).

۴. صنعت و مدیریت:

- تحقیق در عملیات (Operations Research) و بهینه‌سازی سیستم‌ها.

Case Study: چالش‌های علوم انسانی دیجیتال (Digital Humanities)

مورد مطالعاتی: کاربرد هوش مصنوعی در فلسفه اسلامی.

چالش اصلی: متون فلسفی دارای پیچیدگی معنایی و ابهام (Ambiguity) بالایی هستند. مدل‌های زبانی معمولاً روی داده‌های عمومی آموزش دیده‌اند و ممکن است در تفسیر دقیق ترمینولوژی فلسفی دچار "توهم" (Hallucination) شوند.

رویکرد کلاس: بررسی امکان Fine-tune کردن مدل‌ها روی متون تخصصی فلسفی برای دریافت پاسخ‌های استدلالی دقیق.

۶. استراتژی آموزشی: یادگیری جامعه‌محور (Community-Driven Learning)

مدرس دوره بر نقش شرکت‌کنندگان به عنوان "نخبگان" (Elites) جامعه علمی تاکید دارد. بر این اساس، استراتژی کلاس فراتر از یک تدریس یک‌طرفه است:

- **اشتراک‌گذاری دانش (Knowledge Sharing):** افرادی که در حوزه‌های خاص (مثل مخابرات یا پزشکی) تخصص دارند، دانش خود را با سایرین به اشتراک می‌گذارند.
- **یادگیری مبتنی بر هم‌تایان (Peer Learning):** حل مشکلات فنی پروژه‌ها با کمک سایر دانشجویان در گروه کلاسی.
- **اشتراک صفحه نمایش (Screen Sharing):** تغییر متدولوژی کلاس به سمت حل مسئله آنی؛ دانشجویان کدهای خود را روی سرور یا سیستم شخصی اجرا کرده و خطاها را در کلاس برطرف می‌کنند.

نکته مهم: سرعت کلاس تابعی از بازخورد (Feedback) دانشجویان است. با توجه به حجم بالای محتوا، پیش‌فرض بر سرعت بالا است، اما در نقاط کلیدی عملیاتی، سرعت کاهش می‌یابد تا پیاده‌سازی کامل شود.

خلاصه این بخش

- تنوع تکنولوژی در کلاس شامل مدل‌های زبانی (Llama, Gemma) و مدل‌های صوتی پیشرفته (VITS) است.
- کاربردهای LLM فراتر از مهندسی رفته و به حوزه‌هایی مانند فلسفه اسلامی و تحلیل سیگنال‌های پزشکی وارد شده است.
- رویکرد کلاس از "مدرس‌محور" به "جامعه‌محور" تغییر می‌کند و دانشجویان در رفع اشکال یکدیگر نقش فعال دارند.
- ایجاد کانال ارتباطی برای اشتراک‌گذاری منابع و کدهای عملیاتی الزامی شد.

فصل ۱: مبانی فنی و زیرساخت‌های توسعه

۱. نقشه راه جلسه اول

هدف نهایی این جلسه، توانمندسازی دانشجویان برای ساخت یک **Web Application** کامل است که بتواند با مدل‌های زبانی تعامل کند.

- **خروجی مورد انتظار:** ساخت اپلیکیشنی که متن ورودی را دریافت و پاسخ تولید شده توسط LLM را نمایش دهد (Text-to-Text) یا با دریافت تصویر، توضیحات (Caption) مربوطه را تولید کند.
- **ابزار اصلی:** کتابخانه **Gradio** (برای ساخت رابط کاربری سریع و تعاملی)

۲. پیش‌نیازهای برنامه‌نویسی (Python Prerequisites)

برای کار با مدل‌های زبانی و توسعه Agent ها، تسلط بر مفاهیم زیر در زبان پایتون الزامی است:

۱. **توابع (Functions):** توانایی تعریف توابع ماژولار برای فراخوانی مدل‌ها.
۲. **دیکشنری‌ها (Dictionaries):** مدیریت داده‌های کلید-مقدار (Key-Value) که ساختار اصلی پارامترهای مدل هستند.
۳. **JSON (JavaScript Object Notation):** استاندارد تبادل داده در شبکه.

۳. مفهوم API و پروتکل‌های تبادل داده

در توسعه سیستم‌های مبتنی بر LLM، مدل‌ها معمولاً روی سرورهای قدرتمند (مانند OpenAI, OpenRouter) یا سرورهای محلی (اجرا می‌شوند. برنامه ما باید بتواند با این سرورها ارتباط برقرار کند. این ارتباط از طریق **API (Application Programming Interface)** صورت می‌گیرد.

الف) مکانیسم Request/Response

ارتباط با مدل زبانی بر اساس الگوی درخواست (**Request**) و پاسخ (**Response**) است:

۱. **Request:** برنامه ما متنی (Prompt) را بسته‌بندی کرده و به سرور می‌فرستد.
۲. **Processing:** مدل زبانی متن را پردازش می‌کند.
۳. **Response:** سرور پاسخ تولید شده را به برنامه ما برمی‌گرداند.

ب) تفاوت حیاتی: Dictionary vs JSON

یکی از سوالات کلیدی مصاحبه‌های فنی و چالش‌های پیاده‌سازی، درک تفاوت این دو مفهوم است:

- **دیکشنری (Dictionary):** یک ساختار داده (Data Structure) داخلی در حافظه پایتون است. این ساختار فقط در زمان اجرای برنامه و داخل محیط پایتون معنی دارد.
- **جیسون (JSON):** یک فرمت متنی (Text Format/String) استاندارد برای تبادل داده بین سیستم‌های ناهمگن است.

نکته طلایی: وقتی می‌خواهید داده‌ای را از پایتون به یک سرور (که ممکن است با جاوا یا C# نوشته شده باشد) بفرستید، نمی‌توانید "دیکشنری پایتون" بفرستید. باید دیکشنری را به "رشته متنی JSON" تبدیل کنید (Serialization) و در مقصد دوباره آن را به ساختار داده تبدیل کنید (Deserialization).

مثال عملی: تبدیل Dictionary به JSON

در پایتون، این تبدیل با استفاده از کتابخانه json انجام می‌شود:

```
import json

# ۱. این یک دیکشنری پایتون است (در حافظه رم)
user_data = {
    "name": "Ali",
    "role": "Developer",
    "skills": ["Python", "LLM"]
}

# ۲. API برای ارسال به JSON تبدیل دیکشنری به رشته
# (Serialization)
json_payload = json.dumps(user_data)

print(type(user_data))      # <class 'dict'>
print(type(json_payload))   # <class 'str'>
```

۴. استانداردهای OpenAI Compatible API

بسیاری از پلتفرم‌ها و ابزارهای سروینگ مدل (Model Serving) از استاندارد API شرکت OpenAI پیروی می‌کنند. این یعنی اگر شما کدنویسی برای OpenAI را یاد بگیرید، می‌توانید با تغییر آدرس سرور (Base URL) و کلید دسترسی (API Key)، از همان کد برای مدل‌های Llama یا Mistral روی سرور شخصی خودتان استفاده کنید.

خلاصه این بخش

- برای توسعه LLM Application ها، درک عمیق JSON به عنوان زبان مشترک بین سیستم‌ها ضروری است.
- ابزار **Gradio** سریع‌ترین راه برای تبدیل مدل‌های هوش مصنوعی به دموهای قابل تعامل وب است.
- ارتباط با مدل‌ها از طریق ارسال **Request** و دریافت **Response** در قالب JSON انجام می‌شود.
- تمرین عملی این هفته: ساخت یک رابط کاربری ساده با Gradio برای چت کردن با مدل.

فصل ۲: مدیریت داده‌ها و پروتکل‌های ارتباطی

۱. تشریح عمیق ساختار داده‌ها

در توسعه سیستم‌های هوش مصنوعی، درک دقیق نحوه ذخیره‌سازی و انتقال داده‌ها حیاتی است.

الف) دیکشنری پایتون (Python Dictionary)

دیکشنری یک ساختار داده (Data Structure) در حافظه رم است که بر اساس الگوی کلید-مقدار (Key-Value Pair) کار می‌کند.

• ویژگی‌ها:

- کلیدها (Keys) باید یکتا و تغییرناپذیر (Immutable) باشند (معمولاً رشته).
- مقادیر (Values) می‌توانند هر نوع داده‌ای باشند: String, Integer, Boolean, List, یا حتی یک دیکشنری تو در تو.
- دسترسی به مقادیر از طریق کلیدها امکان‌پذیر است (مثلاً `data['first_name']`).

ب) جیسون (JSON - JavaScript Object Notation)

جیسون یک فرمت متنی (String Format) است که ساختاری شبیه به دیکشنری دارد اما صرفاً یک رشته متنی طولانی است.

- کاربرد: استاندارد جهانی برای انتقال داده بین کلاینت (مثلاً مرورگر یا کد پایتون شما) و سرور (API).
- محدودیت: چون JSON فقط متن است، شما نمی‌توانید مستقیماً مثل دیکشنری به مقادیر آن دسترسی داشته باشید (مثلاً `json_data['key']` کار نمی‌کند مگر اینکه اول آن را پارس کنید)

۲. عملیات تبدیل (Serialization & Deserialization)

برای ارتباط با API ها، ما دائماً در حال تبدیل این دو فرمت به یکدیگر هستیم. پایتون کتابخانه استاندارد json را برای این کار ارائه می‌دهد.

توابع کلیدی:

۱. `json.dumps()` : (Dictionary $\rightarrow$ String)
 - کاربرد: تبدیل دیکشنری پایتون به رشته JSON. به این فرآیند **Serialize** کردن می‌گویند.
 - چه زمانی استفاده می‌شود؟ وقتی می‌خواهیم داده‌ای را برای API ارسال کنیم (ارسال Request).
۲. `json.loads()` : (String $\rightarrow$ Dictionary)
 - کاربرد: تبدیل رشته JSON به دیکشنری پایتون. به این فرآیند **Deserialize** کردن می‌گویند.
 - چه زمانی استفاده می‌شود؟ وقتی پاسخی از API دریافت می‌کنیم و می‌خواهیم اطلاعات آن را استخراج کنیم (دریافت Response).

مثال عملی در کلاس (کد پایتون)

```
import json

# ۱. تعریف یک دیکشنری در پایتون (داخل حافظه رم)
# (True/False) با حرف بزرگ شروع می‌شود Boolean توجه کنید: در پایتون
user_profile = {
    "first_name": "Ali",
    "age": ۲۰,
    "is_admin": True, # Python Boolean
    "skills": ["Python", "AI"]
}

print(f"Type before: {type(user_profile)}")
# Output: <class 'dict'>

# ۲. برای ارسال به شبکه (Serialization) JSON تبدیل به
# (با حروف کوچک) تبدیل می‌شود true به True، مقدار JSON توجه کنید: در خروجی
json_string = json.dumps(user_profile)

print(f"JSON Output: {json_string}")
# Output: {"first_name": "Ali", "age": ۲۰, "is_admin": true, "skills":
["Python", "AI"]}

print(f"Type after dumps: {type(json_string)}")
# Output: <class 'str'>  <-- نکته مهم: الان فقط یک رشته متنی است !

# ۳. برای استفاده در برنامه (Deserialization) تبدیل مجدد به دیکشنری
restored_data = json.loads(json_string)
print(restored_data['first_name']) # Output: Ali
```

۳. پاسخ به سوالات دانشجویان (Q&A)

سوال دانشجو: چرا می‌گوییم تایپ JSON استرینگ است، ولی داخلش اعداد (Int) و غیر استرینگ هم وجود دارد؟

پاسخ مدرس: وقتی می‌گوییم تایپ JSON استرینگ است، منظورمان "کل بسته داده" است.

- تصور کنید شما یک نامه می‌نویسید. داخل نامه ممکن است بنویسید "سن من ۲۵ است". عدد ۲۵ داخل نامه وجود دارد، اما ماهیت کلی نامه "کاغذ و جوهر" (متن) است.
- در JSON هم همینطور است. کل `json_string` رشته متنی است. وقتی این رشته را باز می‌کنیم (پارس می‌کنیم)، مفسر می‌فهمد که کاراکترهای ۲ و ۵ کنار هم یعنی عدد ۲۵ و آن را به `Integer` تبدیل می‌کند. اما تا قبل از پارس کردن، همه چیز فقط کاراکترهای متنی هستند.
- نکته ظریف: تفاوت‌های ریز نگارشی وجود دارد. مثلاً در پایتون `True` داریم، اما در JSON استاندارد `true` (با حروف کوچک) داریم. تابع `json.dumps` این تبدیل‌ها را خودکار انجام می‌دهد.

۴. آماده‌سازی محیط Google Colab

برای اجرای کدهای این دوره و جلوگیری از درگیری با مشکلات نصب کتابخانه‌ها روی سیستم‌های شخصی (dependency hell)، از محیط **Google Colab** استفاده خواهیم کرد.

- مزیت: دسترسی رایگان به GPU و محیط ایزوله پایتون.
- نکته: اجرای کدهای `Gradio` (وب‌اپلیکیشن) روی `Colab` نیز ممکن است و لینک عمومی (Public URL) برای تست روی موبایل ارائه می‌دهد.

خلاصه این بخش

- **Dictionary:** ساختار داده داخلی پایتون، قابل دسترسی با کلید، فقط در RAM وجود دارد.
- **JSON:** فرمت متنی استاندارد، برای انتقال در شبکه، باید قبل از استفاده به دیکشنری تبدیل شود.
- تابع `json.dumps()` دیکشنری را به JSON تبدیل می‌کند (مناسب برای ارسال `Request`).
- تابع `json.loads()` رشته JSON را به دیکشنری تبدیل می‌کند (مناسب برای پردازش `Response`).
- محیط توسعه اصلی کلاس **Google Colab** خواهد بود.

فصل دوم (بخش پیشرفته): چالش‌های فرمت داده و تجربیات صنعتی

۵. آینده فرمت‌های داده: آیا JSON جایگزین دارد؟

در دنیای فناوری، هیچ استاندارد ابدی نیست. اخیراً بحث‌هایی در مورد فرمت‌های جدید مانند **TONE** مطرح شده است.

- **فرمت TONE:** یک فرمت داده جدید که ادعا می‌شود "Token Oriented" (توکن‌محور) است. با توجه به اینکه مدل‌های زبانی (LLMs) ذاتاً با توکن‌ها کار می‌کنند، این فرمت شاید سازگاری بهتری با آنها داشته باشد.
- **چالش جایگزینی:** علیرغم مزایای تئوریک TONE، فرمت JSON به دلیل یکپارچگی گسترده با هزاران سیستم موجود (Legacy Systems)، دیتابیس‌ها و API‌های وب، همچنان استاندارد حاکم است. جایگزینی JSON نیازمند تغییر زیرساخت‌های عظیم جهانی است که در کوتاه‌مدت بعید به نظر می‌رسد.

نتیجه‌گیری: فعلاً تمرکز اصلی روی JSON باقی می‌ماند، اما به عنوان متخصص هوش مصنوعی، شناخت ترندهای جدید مثل TONE برای درک آینده تکنولوژی مفید است.

۶. استراتژی ذخیره‌سازی داده‌ها (Data Persistence)

یکی از سوالات مهم در توسعه سیستم‌های هوش مصنوعی، نحوه مدیریت تاریخچه مکالمات است.

چرایی ذخیره‌سازی؟

۱. **حفظ حافظه (Memory):** برای اینکه چت‌بات بتواند به پیام‌های قبلی کاربر ارجاع دهد.
۲. **بهبود مدل (Fine-tuning):** جمع‌آوری جفت‌های "سوال و جواب (Dataset)" برای آموزش مدل‌های اختصاصی در آینده. مثلاً بعد از ۳ ماه فعالیت چت‌بات، شما دیتای ارزشمندی برای ساخت مدل تخصصی خودتان دارید.

روش ذخیره‌سازی

داده‌های JSON (شامل متن سوال کاربر و پاسخ مدل) می‌توانند مستقیماً در پایگاه‌های داده (NoSQL مثل MongoDB یا حتی دیتابیس‌های رابطه‌ای ذخیره شوند تا بعداً بازیابی و تحلیل شوند).

نکات کلیدی این بخش

- استفاده از `str()` برای تبدیل دیکشنری به رشته ممکن است کار کند، اما استاندارد نیست و در مقیاس بالا (Scale) کندتر از `json.dumps()` است.
- ذخیره مکالمات در دیتابیس، سوخت اصلی برای **Fine-tune** کردن مدل‌های آینده شماست.
- تجربه صنعتی نشان می‌دهد که داده‌های واقعی (مثل صدای محیط پرنویز فرودگاه یا لهجه‌های غلیظ) بسیار پیچیده‌تر از داده‌های آزمایشگاهی هستند.

فصل ۳: نقشه راه جامع و جعبه‌ابزار توسعه (Toolbox)

۱. چشم‌انداز دوره و استراتژی آموزشی

با توجه به سطح بالای شرکت کنندگان (Elite Students)، هدف نهایی این دوره فراتر از آموزش تئوری است. تمرکز بر تیم‌سازی (Team-up) و تعریف پروژه‌های واقعی است که پتانسیل تجاری شدن دارند.

تجربه موفق: استارت‌آپ هوشمند مد و فشن

یک نمونه از پروژه‌های موفق قبلی، دستیار هوشمند انتخاب لباس بود که قبل از ظهور ابزارهای جدید (مثل مدل‌های تولید تصویر فعلی) پیاده‌سازی شد:

- **عملکرد:** کاربر عکس خود را آپلود می‌کرد و لباس مورد نظر (مثلاً "لباس شب یلدا با طرح هندوانه") را توصیف می‌کرد.
- **پشت صحنه:** سیستم با استفاده از **Search Engine** در فروشگاه‌های آنلاین (مثل بانی‌مد) جستجو می‌کرد و با **Image Generation**، لباس را روی تن کاربر نمایش می‌داد.

۲. نقشه راه تکنیکال (Technical Roadmap)

مسیری که در ادامه این دوره طی خواهیم کرد، از مفاهیم پایه تا استقرار حرفه‌ای مدل‌ها را پوشش می‌دهد:

گام اول: مبانی و ارتباطات (API & Embedding)

- **درک عمیق Embedding:** چگونه متن به بردار (Vector) تبدیل می‌شود؟
- **کار با OpenAI Compatible APIs:** استانداردسازی ارتباط با مدل‌ها از طریق سرویس‌دهندگانی مثل **OpenRouter**.

گام دوم: توسعه وب‌اپلیکیشن (Web App Development)

- استفاده از **Gradio** برای ساخت سریع دمو.
- استفاده از **Cursor (AI Code Editor)** و تکنیک‌های **AI-Coding** برای نوشتن کدهای پیچیده و اینترفیس‌های زیبا بدون درگیری زیاد با جزئیات فرانت‌اند.

گام سوم: معماری عامل‌گرا (Agentic Workflow)

- آشنایی با **Agents:** سیستم‌هایی که توانایی تفکر، برنامه‌ریزی و استفاده از ابزار دارند.
- استفاده از پلتفرم‌های **Low-Code/No-Code** مانند **n8n** و **Dify** برای مدیریت جریان کار (Workflow) ایجنت‌ها.
- مزیت: نمایش خروجی‌های پیچیده به صورت ویژوال و اتصال راحت به سرویس‌های دیگر.

گام چهارم: مانیتورینگ و بهینه‌سازی (LLMOps)

در سطح سازمانی (Enterprise)، صرفاً ساخت مدل کافی نیست. باید عملکرد آن را رصد کرد.

- ابزارها: **LangFuse, OpenTelemetry**

- متریک‌های کلیدی برای مانیتورینگ:
 - **TTFT (Time To First Token):** چقدر طول می‌کشد تا اولین کلمه پاسخ تولید شود؟ (معیار سرعت پاسخگویی).
 - **Token Usage:** میزان مصرف توکن (هزینه).
 - **Query Analysis:** تحلیل سوالات کاربران برای بهبود مدل در آینده.

چرا مانیتورینگ مهم است؟ (LLMOps)

تصور کنید چت‌باتی برای "گروه خودرویی ماموت" ساخته‌اید. بدون مانیتورینگ، شما کور هستید!

- **مدیریت هزینه:** آیا مدلی که استفاده می‌کنید (مثلاً GPT-4) برای پاسخ دادن به سوالات ساده "قیمت خودرو" زیاده‌روی و گران نیست؟
- **تجربه کاربر:** آیا کاربر ۱۰ ثانیه منتظر می‌ماند تا جواب بگیرد؟ (TTFT) بالا.
- **بهبود مستمر:** کاربران چه سوالاتی می‌پرسند که مدل ما بلد نیست جواب دهد؟

با ابزارهایی مثل **LangFuse**، تمام این داده‌ها داشبورد می‌شوند تا بتوانید فرآیند **Fine-tuning** بعدی را دقیق‌تر انجام دهید.

برنامه جلسه بعد (شروع عملی)

پس از استراحت و صرف صبحانه، مستقیماً وارد محیط کدنویسی می‌شویم:

۱. دریافت API Key از پرووایدرها.
۲. نوشتن اولین اسکریپت پایتون برای ارسال Request.
۳. ساخت رابط کاربری با Gradio.

فصل ۴: اتوماسیون هوشمند با n^n و انتخاب محیط توسعه

۱. معرفی ابزار (n^n Workflow Automation)

یکی از ابزارهای قدرتمند که امروزه در کنار کدنویسی پایتون برای ساخت **Agent**‌ها استفاده می‌شود، پلتفرم n^n است.

- چرا n^n ؟ چون به شما اجازه می‌دهد جریان‌های کاری (Workflows) پیچیده را به صورت بصری (Visual) طراحی کنید.
- نحوه کارکرد: این ابزار بر اساس **Node**‌ها کار می‌کند. هر Node یک وظیفه خاص دارد (مثلاً دریافت پیام، پردازش توسط AI، ارسال به تلگرام).

ساخت یک ایجنت ساده در n^n مثال عملی کلاس)

در این مثال، می‌خواهیم جریانی بسازیم که کاربر سوالی بپرسد، مدل پاسخ دهد و نتیجه به تلگرام ارسال شود:

۱. **Trigger Node:** شروع‌کننده جریان (مثلاً دریافت پیام در چت یا وب‌هوک).
 ۲. **LLM Node:** اتصال به مدل زبانی (مثلاً از طریق **OpenRouter**).
 - تنظیمات: انتخاب مدل (مثلاً ۳, Llama یا ۲, Gemma یا GPT-۴o mini).
 - ورودی: متن پیام کاربر.
 ۳. **Output Node:** ارسال پاسخ به مقصد (مثلاً نود تلگرام).
 - اتصال داده: در اینجا مفهوم **JSON** و **Key-Value** که قبلاً یاد گرفتیم حیاتی است. خروجی نود LLM یک JSON است که ما باید مقدار داخل کلید content یا text را برداریم و به نود تلگرام بدهیم.
- نکته تجربی: در n^n شما دقیقاً می‌بینید که خروجی هر نود یک JSON است. این به درک بهتر جریان داده کمک می‌کند.

۲. بررسی مدل‌های زبانی در OpenRouter

پلتفرم **OpenRouter** به عنوان یک هاب (Hub) عمل می‌کند که دسترسی به صدها مدل زبانی را فراهم می‌کند. در دمو کلاس دیدیم که می‌توانیم مدل‌های مختلف را تست کنیم:

- مدل‌های اختصاصی: (Proprietary) مثل Claude ۳.۵ Sonnet, GPT-۴o.
- مدل‌های متن‌باز: (Open Source) مثل Llama ۳, Qwen, Gemma ۲.
- مدل‌های بهینه: (Distilled/Small) مثل GPT-۴o mini یا مدل‌های Flash که برای سرعت و هزینه کم بهینه شده‌اند.

۳. جنگ محیط‌های توسعه (IDE Wars)

برای توسعه پروژه‌های هوش مصنوعی، کدام محیط مناسب‌تر است؟

الف) Visual Studio Code (VS Code)

- وضعیت: محبوب‌ترین ادیتور حال حاضر جهان.
- مزایا:
 - پشتیبانی از چند زبان (Python, JS, HTML) در یک محیط.
 - اکستنشن‌های فراوان از جمله اکستنشن جدید. (Google Colab)
 - مناسب برای پروژه‌های بزرگ و Agent یک که ترکیبی از Backend و Frontend هستند.

ب) PyCharm

- وضعیت: IDE تخصصی و قدرتمند پایتون.

- **چالش:** کمی سنگین است و شاید برای پروژه‌های مدرن که نیاز به تعامل با جاوااسکریپت و وب دارند، به انعطاف‌پذیری VS Code نباشد). به گفته مدرس: PyCharm "یه خورده دور افتاد و VS Code بازی رو خورد."!

ج) Jupyter Notebook / Google Colab

- **وضعیت:** استاندارد آموزشی و آزمایشگاهی. (Data Science)
- **مزایا:**
 - اجرای خط به خط کد و مشاهده آنی خروجی. (Visual)
 - ایده‌آل برای آموزش و تست اولیه مدل‌ها.
 - **خبر خوب:** اکنون می‌توانید از اکستنشن رسمی Google Colab داخل VS Code استفاده کنید و بهترین‌های هر دو دنیا را داشته باشید.

۴. پرسش و پاسخ: نقش RLHF

سوال: آیا در این دوره به **RLHF (Reinforcement Learning from Human Feedback)** می‌پردازیم؟ **پاسخ:** در مبحث Agent ها، تمرکز اصلی ما روی **RL (Reinforcement Learning)** نیست. اما به RLHF (یادگیری تقویتی از بازخورد انسانی) اشاراتی خواهیم کرد، زیرا این تکنیک روش اصلی برای "همسوسازی (Alignment)" مدل‌های زبانی است تا پاسخ‌های مفید و بی‌خطر بدهند.

خلاصه این بخش

- ابزار **nan** راهی سریع برای ساخت Workflow های هوشمند بدون کدنویسی سنگین است.
- در **nan**، انتقال داده بین نودها دقیقاً بر اساس ساختار **JSON** انجام می‌شود.
- برای توسعه، **VS Code** به دلیل اکوسیستم جامع و افزونه‌هایش توصیه می‌شود، اما برای کلاس از **Google Colab** استفاده می‌کنیم.
- سایت **OpenRouter** به ما اجازه می‌دهد بدون تغییر کد، مدل‌های مختلف (از متا گرفته تا گوگل و آنتروپیک) را تست کنیم.

فصل ۵: استقرار محلی و مقایسه اکوسیستم‌های توسعه

۱. اجرای مدل‌های زبانی به صورت محلی (Local Hosting)

بسیاری از سازمان‌های بزرگ (بانک‌ها، اپراتورها و نهادهای دولتی) به دلایل امنیتی یا قطع اینترنت، امکان ارسال داده‌های حساس به سرورهای خارجی مثل OpenAI را ندارند. راه حل چیست؟

ابزار Ollama

- چیست؟ ابزاری ساده و قدرتمند برای اجرای مدل‌های متن‌باز (Open Source) مانند Llama ۳ روی سیستم شخصی یا سرور داخلی.
- کاربرد: ایجاد یک API محلی (Local API) که دقیقاً مشابه OpenAI کار می‌کند، اما داده‌ها هرگز از شبکه داخلی سازمان خارج نمی‌شوند.
- مزیت: برای مدیران و دموهای سریع، مثل یک "اسباب‌بازی جذاب (Tech Toy)" عمل می‌کند که قدرت هوش مصنوعی را بدون پیچیدگی نشان می‌دهد.
- تمرین: برای جلسه آینده، نصب Ollama روی سیستم‌های شخصی توصیه می‌شود.

۲. سناریوی عملی: دستیار صوتی هوشمند شخصی

در کلاس، یک سناریوی کامل اتوماسیون با n^n بررسی شد:

۱. ورودی: ارسال یک پیام صوتی (Voice) در تلگرام: "فردا چه جلساتی داریم؟"
 ۲. پردازش:
 - فایل صوتی توسط مدل **Whisper** (یا مشابه) به متن تبدیل می‌شود. (Transcribe)
 - متن به یک LLM داده می‌شود تا قصد کاربر (Intent) را بفهمد.
 ۳. اقدام (Action): ایجنت به **Google Calendar** متصل می‌شود و رویدادهای فردا را می‌خواند.
 ۴. خروجی: لیست جلسات به صورت متنی در تلگرام به کاربر برگردانده می‌شود.
- چالش فنی: برای اتصال گوگل کلندر به n^n لوکال، گوگل نیاز به دامنه امن (HTTPS) دارد. در اینجا نیاز به تنظیمات VPS, Docker, Nginx و احتمالاً استفاده از Cloudflare برای تونل زدن داریم تا گوگل بتواند به سرور ما متصل شود.

۳. جنگ ابزارهای اتوماسیون: n^n در برابر Flowise

یکی از دانشجویان (هادی) به ابزار **Flowise** اشاره کرد که منجر به مقایسه‌ای ارزشمند شد:

ویژگی	n^n	Flowise
تمرکز	اتوماسیون کلی (General Workflow Automation)	تخصصی برای LLM بر پایه (LangChain)
جامعه کاربری (Community)	بسیار بزرگ و فعال	رو به رشد اما کوچکتر
رابط کاربری (UI)	کاربردی و ماژولار	زیباتر و مدرن (React-based)
لایسنس (License)	(Sustainable Use License) محدودیت‌هایی برای استفاده تجاری خاص دارد	(Apache ۲.۰) کاملاً متن‌باز و آزاد برای هر نوع استفاده تجاری
کاربرد سازمانی	مناسب برای اتصال هزاران سرویس مختلف	مناسب برای ساخت سریع چت‌بات‌های RAG و ایجنت

تجربه پروژه بانکی (Case Study)

در پروژه "بانک ملیورن" (یا مشابه)، تیم فنی از **Flowise** استفاده کردند. دلیل اصلی این انتخاب، لایسنس آزاد (Apache ۲.۰) و رابط کاربری تمیز آن بود که به کارفرما اجازه می‌داد تغییرات را راحت‌تر اعمال کند.

ایده استارت‌آپی: با توجه به اینکه Flowise متن‌باز است، می‌توان با فارسی‌سازی و بومی‌سازی آن، یک "سکوی ملی طراحی چت‌بات" برای سازمان‌های داخلی توسعه داد.

جمع‌بندی نکات فنی

- برای پروژه‌های شخصی و یادگیری، نسخه ابری رایگان ۱۴ روزه n^n بهترین گزینه است.
- اگر در سازمانی هستید که دغدغه حریم خصوصی دارد، ترکیب **Ollama + Flowise** روی سرور داخلی، قدرتمندترین راهکار امن است.
- در بحث اتصال سرویس‌های خارجی (مثل Google)، دانش زیرساختی (VPS, DNS, SSL) ضروری است.

فصل ۶: چالش‌های زیرساختی و سخت‌افزاری

۱. پلتفرم Dify و مساله لایسنسینگ

علاوه بر Flowise، ابزار دیگری به نام **Dify** نیز وجود دارد که برای ساخت چت‌بات بسیار محبوب است.

- **مزیت Dify:** رابط کاربری بسیار زیبا و User-Friendly دارد که ساخت ایجنت‌ها را لذت‌بخش می‌کند.
- **چالش حقوقی (Legal):** در پروژه‌های ملی (مثل پروژه معاونت علمی)، لایسنس نرم‌افزار بسیار مهم است. وقتی قرار است پروژه‌ای به عنوان "متن‌باز ملی" منتشر شود، باید بر پایه لایسنس‌های آزاد (مثل Apache ۲.۰) باشد.
- مثال: شرکت‌های بزرگ (مثل ایرانسل یا ماموت) می‌توانند در داخل شبکه خود از مدل Llama استفاده کنند، اما نمی‌توانند محصولی بفروشند که هسته اصلی آن مدل Llama باشد و قوانین متا را نقض کند.

۲. روش‌های استقرار (Deployment Strategies)

برای اجرای ابزارهایی مثل n^n یا Flowise، سه راهکار اصلی وجود دارد:

الف) استفاده از Cloud نسخه ابری

- **روش:** ثبت نام در سایت n^n.io نسخه ۱۴ روزه رایگان.
- **مزیت:** سریع‌ترین راه برای یادگیری و تست، بدون نیاز به نصب.
- **مناسب برای:** شروع کلاس و تمرینات هفته اول.

ب) سرور شخصی (Localhost / PC)

• چالش‌ها:

- لپ‌تاپ باید همیشه روشن باشد (مثلاً برای ایجنتی که ساعت ۷ صبح باید گزارش دهد).
- مشکلات فیلترینگ (اتصال به تلگرام و گوگل در دسترس است).
- نیاز به تنظیمات پیچیده شبکه و پروکسی.

ج) سرور مجازی خارجی (Foreign VPS)

• روش: خرید یک VPS خارج از ایران.

• مزیت:

- فیلتر نیست (دسترسی آزاد به تلگرام و API های هوش مصنوعی).
- IP ثابت دارد (مناسب برای تنظیمات DNS و SSL).
- همیشه روشن است. (Reliability)
- توصیه مدرس: برای پروژه‌های جدی، خرید یک VPS ارزان‌قیمت بهترین سرمایه‌گذاری است.

۳. محاسبات و سخت‌افزار (GPU Requirements)

چرا برای اجرای مدل‌ها به GPU نیاز داریم؟

درک پارامترهای مدل

وقتی می‌گوییم مدلی ۸B (۸ میلیارد) پارامتر دارد، یعنی یک شبکه عصبی با ۸ میلیارد "وزن" (Weight) داریم.

- تشبیه: این وزن‌ها مثل پیچ‌های تنظیم یک رادیو هستند. در فرآیند آموزش (Training)، این پیچ‌ها تنظیم شده‌اند.
- نقش **VRAM**: برای اینکه مدل بتواند کار کند (Inference)، باید تمام این ۸ میلیارد عدد را در حافظه کارت گرافیک بارگذاری کنیم.
- محاسبه سرانگشتی: هر پارامتر معمولاً ۲ یا ۴ بایت فضا می‌گیرد.
 - مدل ۸ میلیاردی با دقت $\approx \$16 \text{ fp16}$ گیگابایت VRAM نیاز دارد.
 - با تکنیک **Quantization** (کاهش دقت)، می‌توان این حجم را به ۴-۵ گیگابایت رساند تا روی کارت‌های گیمینگ معمولی هم اجرا شود.

جنگ ابزارهای سروینگ: Ollama vs vLLM

یکی از دانشجویان (طاها) سوال فنی دقیقی پرسید.

- **Ollama**: مثل "دوچرخه" است. عالی برای شروع، کاربری شخصی و تست روی لپ‌تاپ.
- **vLLM**: مثل "ماشین مسابقه" است. راهکار سازمانی (Enterprise) برای سرویس‌دهی به هزاران کاربر همزمان. سرعت پردازش توکن (Throughput) در vLLM بسیار بالاتر است.

۴. مدل‌های تبدیل متن به گفتار (TTS) فارسی

برای پروژه‌های صوتی فارسی، گزینه‌ها محدود اما رو به رشد هستند:

۱. **ElevenLabs**: بهترین کیفیت جهانی (پولی). حتی از متن فارسی هم خروجی فوق‌العاده‌ای می‌دهد.
 ۲. **RVC (Retrieval-based Voice Conversion)**: تکنولوژی تغییر صدا (مثلاً تبدیل صدای خواننده به هاید). کیفیت بالایی دارد اما بیشتر برای تغییر صداست تا تولید گفتار.
 ۳. **Coqui TTS**: گزینه متن‌باز خوب که روی CPU هم اجرا می‌شود (مناسب سرورهای بدون GPU).
 ۴. **MMS (Massively Multilingual Speech)**: مدل فیس‌بوک (Meta) که زبان‌های زیادی را پشتیبانی می‌کند.
- نکته: مدرس قول داد محتوای این دوره بسیار فراتر از دوره‌های مرسوم (مثل مکتب‌خونه) باشد و وارد جزئیات فنی عمیق شود.

اقدامک‌های ضروری

- فعلاً نیازی به نصب Ollama یا Docker روی سیستم شخصی نیست؛ تمرینات امروز روی Cloud انجام می‌شود.
- اگر قصد پروژه جدی دارید، به فکر تهیه یک VPS خارجی باشید.
- اکانت رایگان nⁿ را بسازید.

فصل ۷: جمع‌بندی ساعت اول و آمادگی برای کدنویسی

۱. ارزیابی نهایی: nⁿ اسباب‌بازی یا ابزار تجاری؟

آیا می‌توان روی nⁿ به عنوان یک محصول نهایی حساب کرد؟

- برای **POC (Proof of Concept)** فوق‌العاده است. برای اینکه به کارفرما یا سرمایه‌گذار نشان دهید ایده شما کار می‌کند، بهترین گزینه است.
- برای استفاده داخلی (**Internal Tool**): کاملاً جواب می‌دهد. مثلاً برای شرکتی که ۲۰-۳۰ کارمند دارد و می‌خواهد فرآیندهای داخلی را خودکار کند.
- برای **Enterprise Product**: اگر بخواهید سرویسی عمومی بسازید که به هزاران مشتری سرویس دهد، nⁿ انتخاب مناسبی نیست (به دلیل محدودیت‌های مقیاس‌پذیری و هزینه). در این حالت، کدنویسی اختصاصی (Custom Code) با Python/Go و استفاده از معماری میکروسرویس منطقی‌تر است.

۲. برنامه ادامه کلاس (نقشه راه امروز)

بعد از استراحت کوتاه، وارد فاز کاملاً عملی و کدنویسی خواهیم شد. سرفصل‌های پیش رو:

۱. بررسی ساختار **API**: درک پروتکل HTTP، متدهای POST/GET و نحوه ارسال درخواست به مدل.
۲. کار با **OpenAI Compatible API**: یادگیری استاندارد جهانی ارتباط با LLM ها.
۳. ساخت **Web Application**: استفاده از پایتون برای ساخت اولین چت‌بات تعاملی.
۴. معماری **RAG** اشاره‌مقدماتی: (بحث وکتور دیتابیس‌ها (Vector DB) و تکنیک RAG احتمالاً یک جلسه کامل (۸ ساعت) را در هفته‌های آینده به خود اختصاص دهد، اما مقدمات آن را بررسی می‌کنیم.

۳. منابع و فایل‌های اشتراکی

مدرس فایل‌های زیر را برای دانشجویان آماده کرده است:

- اسلایدها: (PDF) فایل ارائه جامع. "LLM Agent - Session ۰۱"
- پروژه‌های نمونه: فایل‌های JSON مربوط به Workflow های n^n که در کلاس دمو شد (اتصال تلگرام و تقویم).
- ضبط کلاس: ویدیوها پارت‌بندی شده و در گروه قرار می‌گیرد.

پایان بخش اول (تئوری و زیرساخت)

کلاس برای استراحت صبحگاهی (۱۰:۳۰ تا ۱۰:۴۵) متوقف می‌شود.

فصل ۸: استراتژی یادگیری و کدنویسی مدرن

۱. کدنویسی در عصر AI: آیا باید خودمان کد بزنیم؟

با ظهور ابزارهایی مثل ChatGPT و Copilot، رویکرد برنامه‌نویسان تغییر کرده است.

- تولید کد: (Generation) بسیاری از کدهای Boilerplate (کدهای تکراری و ساختاری) را می‌توان به AI سپرد.
- مسئولیت برنامه‌نویس: وظیفه اصلی ما از "نوشتن کد" به "خواندن، دیباگ کردن و درک کد" تغییر کرده است.
 - خطر: اگر کدی را که AI تولید کرده بدون درک کردن کپی-پیست کنید ("Copy-Paste")
 - در اولین باگ گیر خواهید کرد. ("Programming")
 - روش صحیح: کد را بگیرید، خط به خط بخوانید، منطق آن را بفهمید و سپس استفاده کنید. این همان "نظارت" (Supervision) است که انسان باید انجام دهد.

نکته کلیدی: قبلاً با زدن یک (Dot) . لیست متدها را می‌دیدیم. (IntelliSense) الان کل فانکشن برای ما جنریت می‌شود. این افزایش سرعت است، اما نباید باعث کاهش دانش پایه شود.

۲. چرا کلاس حضوری/آنلاین؟ (فلسفه دوره)

در دنیایی که همه آموزش‌ها در یوتیوب هست، چرا باید وقت و هزینه صرف کلاس کرد؟

۱. **سرعت انتقال تجربه:** هدف این است که مسیری که مدرس در ۱۰۰ ساعت طی کرده و "به دیوار خورده"، دانشجو در ۲۰ ساعت و بدون خطا طی کند.

۲. **یادگیری فعال (Active Learning):**

- **قانون طلایی:** "فرض کنید ویدیویی در کار نیست!"
- یادگیری باید همین لحظه در کلاس اتفاق بیفتد. موکول کردن به "بعداً ویدیو را می‌بینم" معمولاً به معنی "هرگز یاد نخواهم گرفت" است.
- اگر سوالی دارید، همان لحظه پرسید. تعامل زنده ارزش اصلی کلاس است.

۳. **شبکه‌سازی (Community):** آشنایی با هم‌کلاسی‌هایی که هر کدام در حوزه‌ای (مثل خودرو، بانک، شبکه) متخصص هستند.

۳. نقش Deep Learning در این دوره

یکی از دانشجویان (امید) پرسید که چقدر باید دیپ لرنینگ (Deep Learning) بلد باشیم؟

- **پاسخ:** در این دوره نیازی نیست که شما متخصص ریاضیات دیپ لرنینگ باشید یا بتوانید شبکه عصبی طراحی کنید.
- تمرکز ما بر کاربرد (Application) و مهندسی (Engineering) مدل‌هاست، نه تحقیق و توسعه (R&D) الگوریتم‌های پایه. با این حال، درک مفاهیم اولیه (مثل اینکه "وزن" چیست یا "توکن" چگونه کار می‌کند) برای دیباگ کردن مدل‌ها ضروری است.

توصیه اکید مدرس

"همه چیز در یوتیوب با موزیک و افکت‌های جذاب هست! شما اینجا هستید تا **عمل کنید**. تا وقتی خودتان دست به کد نشوید و ارور نگیرید، چیزی یاد نمی‌گیرید. پس لطفاً همین الان IDE خود را باز کنید."

فصل ۹: هنر یکپارچه‌سازی و معماری مدل‌های زبانی

۱. بزرگترین چالش فعلی: اینتگریشن (Integration)

امروز ساختن یک چت‌بات ساده (مثل ChatGPT) کار سختی نیست و دیگر مزیت رقابتی محسوب نمی‌شود. چالش اصلی و جایی که پول در آن نهفته است، **یکپارچه‌سازی (Integration)** هوش مصنوعی با فرآیندهای واقعی کسب‌وکارهای موجود است.

سناریوهای واقعی در محیط کار:

- مثال منشی شرکت: منشی هر روز نامه‌هایی با غلط املایی و نگارشی تایپ می‌کند. مدیر مجبور است وقت بگذارد و اصلاح کند.
 - راه‌حل: یک اتوماسیون ساده که متن را از منشی می‌گیرد، با LLM اصلاح می‌کند و روی سربرگ رسمی شرکت قرار می‌دهد.
 - مثال ماموت خودرو: کارمندان درگیر کارهای تکراری هستند (مثلاً ۸ ساعت کار می‌کنند اما ۱ ساعت آن صرف کارهای بیهوده مثل پر کردن فرم تکراری می‌شود).
 - راه‌حل: شناسایی گلوگاه‌ها (Pain Points) و حل آن‌ها با یک اسکریپت ساده AI.
 - مثال مارکتینگ: تولید ویدیوی "کارمند نمونه" در ۱۰ ثانیه! عکس پرسنل را می‌گیرد و با هوش مصنوعی متحرک می‌کند.
- درس بیزینسی: شما باید "بیزینس" را بشناسید. دانش فنی به تنهایی کافی نیست. باید بدانید در بانک، کارخانه یا شرکت بیمه، دقیقاً کجا یک "گره" وجود دارد که AI می‌تواند آن را باز کند.

۲. مبانی فنی مدل‌های زبانی (LLM Foundations)

حالا که چرایی استفاده را فهمیدیم، بیایید ببینیم زیر کاپوت چه خبر است.

همه چیز عدد است (Everything is Numbers)

در دنیای کامپیوتر و هوش مصنوعی، ما با چیزی جز عدد سروکار نداریم.

- متن، تصویر، صدا و ویدیو، همگی باید به بردار (Vector) یا ماتریس‌های عددی تبدیل شوند تا ماشین بتواند آن‌ها را بفهمد.
- Embedding (امبدینگ): فرآیند تبدیل داده (مثل کلمه "سیب") به یک بردار عددی (مثلاً $[0.1, 0.5, -0.2]$) که معنای آن را در خود دارد.

جایگاه LLM در اکوسیستم هوش مصنوعی

۱. Artificial Intelligence (AI): هوشمندی تزریق شده به ماشین.
۲. Machine Learning (ML): استفاده از آمار و احتمال برای یادگیری الگوها (Patterns) از داده‌ها.
۳. Deep Learning (DL): استفاده از شبکه‌های عصبی (Neural Networks) چند لایه.
۴. Generative AI & LLMs: زیرمجموعه‌ای از دیپ لرنینگ که بر پایه معماری خاصی به نام Transformer بنا شده است.

انقلاب ترنسفورمرها (The Transformer Revolution)

- ژوئن ۲۰۱۷: گوگل مقاله تاریخی "Attention Is All You Need" را منتشر کرد. این مقاله معماری Transformer را معرفی کرد.

- معماری: شامل دو بخش اصلی **Encoder** (کدگذار) و **Decoder** (کدگشا) است.
- مکانیزم توجه: (**Attention Mechanism**) کلید موفقیت این مدل هاست. به مدل اجازه می‌دهد روی بخش‌های مهم ورودی تمرکز کند (مثل توجه انسان).
- **GPT: حرف T** در **GPT** مخفف **Transformer** است.
- مسیر تکامل: BERT (گوگل) -> GPT-۳ -> GPT-۲ -> GPT-۱ -> ChatGPT (نوامبر ۲۰۲۲).

نکته آموزشی

درک معماری Transformer (انکودر/دیکودر) برای دیباگ کردن مدل‌ها در آینده حیاتی است. وقتی مدل شما "توهم" می‌زند (Hallucination) یا کند کار می‌کند، ریشه در همین معماری و نحوه پردازش توکن‌ها دارد.

فصل ۱۰: مکانیک مدل‌های زبانی؛ توکن‌ها، احتمالات و پارامتر دما

۱. چرا ترنسفورمرها (Transformers) مهم هستند؟

امروزه همه چیز در دنیای هوش مصنوعی مولد (Generative AI) بر مدار معماری **Transformer** می‌چرخد.

- **سلطه مطلق:** تمام مدل‌های مشهور (GPT, BERT, Llama, Mistral) و حتی سرویس‌های جدید تبدیل متن به ویدیو، مبتنی بر ترنسفورمر هستند.
- **آینده:** شاید روزی معماری جدیدی مثل **Mamba** جایگزین شود، اما فعلاً و احتمالاً تا سال‌های آینده، "بازی روی ترنسفورمرهاست".
- **پیش‌نیاز:** برای درک اینکه چرا مدل‌های زبانی گاهی کند هستند یا چرا حافظه (Context) محدودی دارند، باید بدانید **Attention Mechanism** (مکانیزم توجه) چگونه کار می‌کند.

۲. زبان ماشین: توکن (Token) و عدد

مدل زبانی نه کلمه می‌فهمد، نه جمله و نه معنی. تنها چیزی که می‌فهمد **Token** و عدد است.

فرآیند پردازش متن:

۱. **Tokenization:** متن ورودی (مثلاً "The cat likes") خرد می‌شود. هر کلمه یا بخشی از کلمه تبدیل به یک توکن می‌شود.
۲. **ID Assignment:** به هر توکن یک شناسه عددی (Token ID) اختصاص می‌یابد (مثلاً ۱۰۱ -> The).
۳. **Embedding:** این اعداد به بردارهای ریاضی تبدیل می‌شوند تا مدل بتواند روی آن‌ها محاسبات انجام دهد.

۳. پیش‌بینی کلمه بعدی (Next Token Prediction)

هسته مرکزی تمام LLM ها یک اصل ساده است: "بر اساس توکن‌های قبلی، توکن بعدی چه احتمالی دارد؟"

• مثال: جمله "The capital city of France is..."

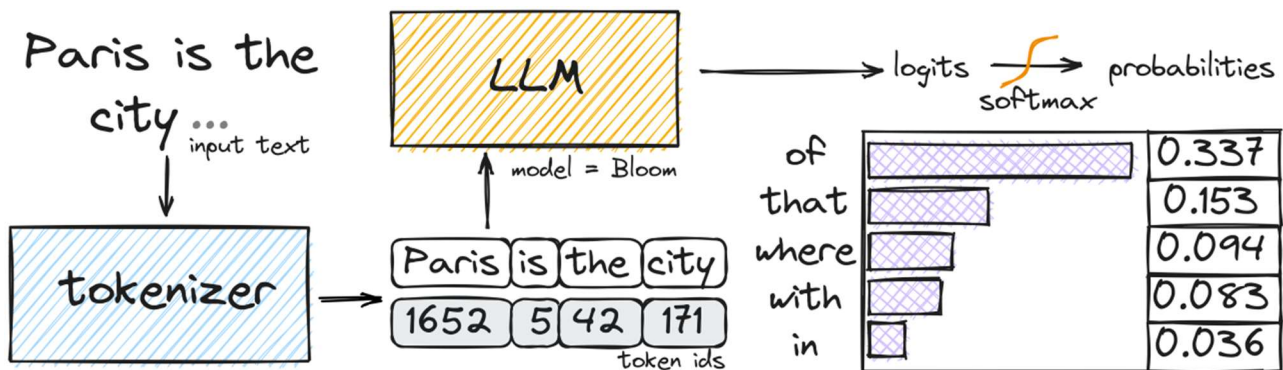
• احتمالات مدل:

○ Paris: ۹۹.۸٪

○ Lyon: ۰.۱٪

○ Tehran: ۰.۰۰۱٪

مدل بر اساس آمار و احتمالات (Probability & Statistics) که در زمان آموزش یاد گرفته، محتمل‌ترین گزینه را انتخاب می‌کند.



۴. پارامتر Temperature: کنترل خلاقیت

چگونه می‌توانیم رفتار مدل را کنترل کنیم تا گاهی دقیق باشد و گاهی خلاق؟ با پارامتری به نام Temperature دما.

• Temperature پایین (نزدیک به ۰):

○ مدل همیشه "محتمل‌ترین" گزینه را انتخاب می‌کند.

○ نتیجه: پاسخ‌های قطعی، تکراری و دقیق (مناسب برای کدنویسی یا کارهای بانکی).

○ مثال: همیشه می‌گوید "Paris".

• Temperature بالا (نزدیک به ۱):

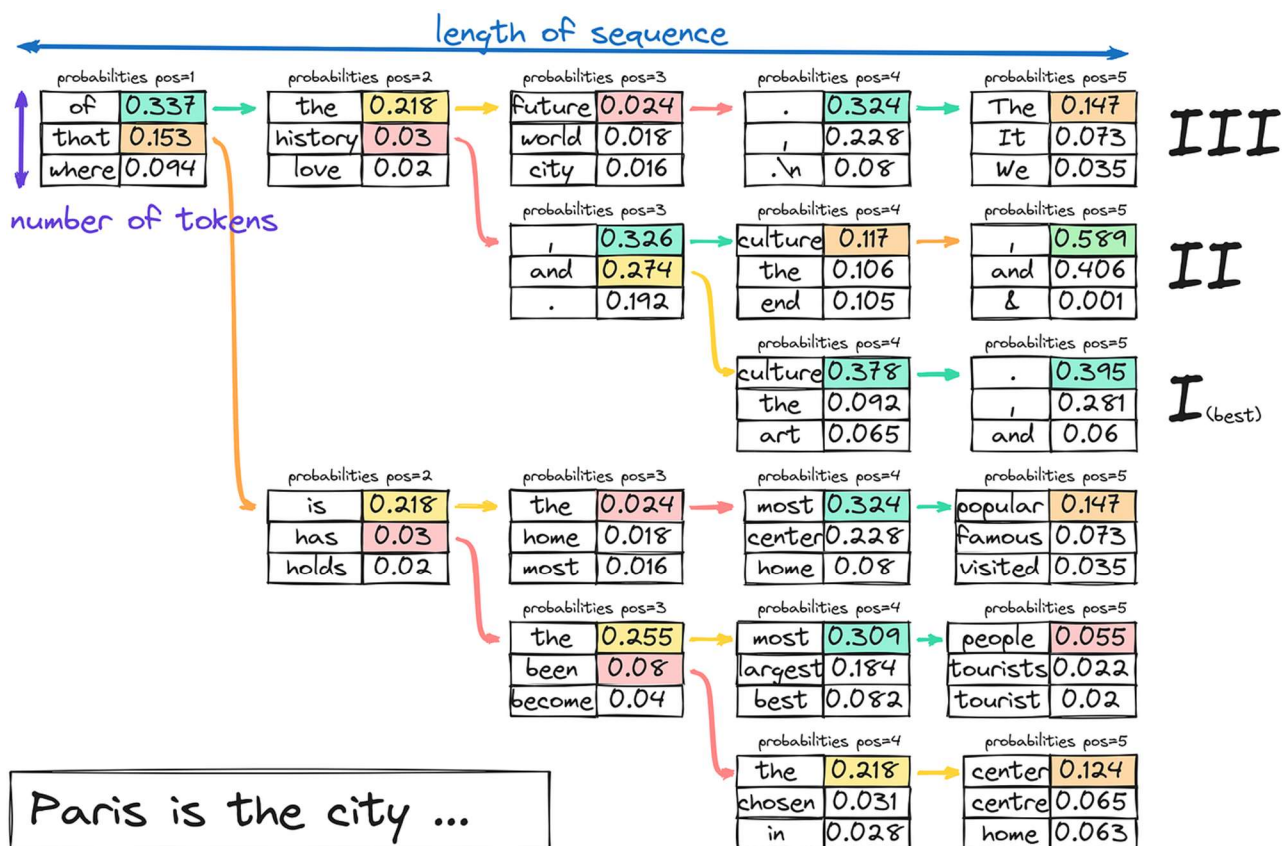
○ مدل ریسک می‌کند و گزینه‌های با احتمال کمتر را هم انتخاب می‌کند.

○ نتیجه: پاسخ‌های متنوع، خلاقانه و گاهی عجیب (مناسب برای نوشتن داستان یا شعر).

○ مثال: ممکن است بگوید "Lyon" یا حتی یک پاسخ شاعرانه عجیب.

○ تعبیر مدرس: "انگار مدل نشسته پای بساط و فضا عوض می‌شه!" باعث توهم یا Hallucination می‌شود.

نکته کلیدی: تفاوت اینکه هر بار از ChatGPT سوال می‌کنید و جواب متفاوتی می‌گیرید، در همین ماهیت احتمالاتی (Probabilistic) و تنظیمات Temperature نهفته است. بر خلاف برنامه‌نویسی سنتی (Rule-based) که همیشه $\text{if } A \text{ then } B$ است، اینجا $\text{if } A \text{ then probably } B, \text{ maybe } C$ است.



۵. اندازه مدل: ۴ میلیارد vs ۶۷۰ میلیارد پارامتر

تفاوت یک مدل کوچک (مثل ۸B - Llama ۳ با یک غول) مثل ۶۷۱B - DeepSeek در چیست؟

۱. حجم دانش: مدل بزرگتر کتاب‌های بیشتری خوانده است.
۲. قدرت استدلال: مدل بزرگتر روابط پیچیده‌تری بین توکن‌ها کشف کرده است.
۳. خلاقیت: مدل ۶۷۱ میلیاردی فضای احتمالاتی غنی‌تری دارد و می‌تواند جملات پیچیده‌تر و خلاقانه‌تری تولید کند.

جمع‌بندی فنی

- مدل زبانی یک ماشین حساب احتمالات است، نه یک موجود هوشمند.
- Temperature آهرم شما برای تنظیم بین "دقت" و "خلاقیت" است.
- همه چیز در نهایت به Token ID تبدیل می‌شود.

فصل ۱۱: جنگ مدل‌ها و معماری مسیریابی (Routing)

۱. مدل بزرگ (LLM) یا مدل کوچک (SLM)؟

آیا همیشه "بزرگتر" بهتر است؟ مقایسه مدل ۶۷۰ میلیاردی با مدل ۴ میلیاردی:

ویژگی	مدل بزرگ (LLM - e.g., DeepSeek) (۶۷۱B)	مدل کوچک (SLM - e.g., Phi-۳, Llama) (۸B-۳B)
ظرفیت مغز	عظیم (۶۷۰ میلیارد نورون/پارامتر)	محدود اما چابک
هزینه زیرساخت	بسیار بالا (نیاز به چندین GPU A۱۰۰، کولینگ، برق)	پایین (قابل اجرا روی لپ‌تاپ یا سرور ارزان)
سرعت	کندتر (به دلیل حجم محاسبات)	بسیار سریع
کاربرد	کارهای پیچیده، استدلال عمیق، خلاقیت بالا	کارهای مشخص، طبقه‌بندی، ابزارهای داخلی

نتیجه‌گیری تجاری: برای اکثر پروژه‌های واقعی (Product)، مدل‌های SLM (Small Language Models) گزینه‌های بهتری هستند. مدل‌های غول‌پیکر فقط برای کارهای خاص یا به عنوان "معلم" برای مدل‌های کوچکتر استفاده می‌شوند.

۲. چالش اصلی: توهم (Hallucination)

مدل زبانی فقط یک "طوطی هوشمند" است که کلمات را بر اساس احتمالات تکرار می‌کند. هیچ درک واقعی (Reasoning) از جهان ندارد) هرچند مدل‌های جدید مثل GPT-۵ در حال بهبود این ضعف هستند).

- خطر: اگر مدل را به حال خود رها کنید، ممکن است با اعتماد به نفس کامل دروغ بگوید (مثلاً داروی اشتباه تجویز کند).

راهکارهای مقابله با توهم:

۱. Fine-tuning تنظیم دقیق: (آموزش مجدد مدل با داده‌های تخصصی خودتان. (مثل فرستادن دانشجو به دوره

تخصصی پزشکی).

۲. RAG (Retrieval-Augmented Generation): اتصال مدل به یک "منبع دانش بیرونی" (مثل کتابخانه).

- روش کار: مدل قبل از جواب دادن، اول در دیتابیس شما جستجو می‌کند، متن مرتبط را می‌خواند و بر اساس آن جواب می‌دهد.

- مزیت: ارزان‌تر، دقیق‌تر و قابل‌اعتمادتر از Fine-tuning برای جلوگیری از توهم.



محدودیت: چت‌بات حقیقت را نمی‌فهمد؛ ممکن است اشتباه کند یا اطلاعات ساختگی تولید کند.

نکات ایمنی:

- اطلاعات محرمانه سازمان را وارد نکنید.
- حتماً خروجی‌ها را بازبینی کنید.
- برای تصمیم‌های مهم فقط به چت‌بات اعتماد نکنید.



۳. معماری هوشمند: LLM Router

چگونه یک سیستم جامع بسازیم که هم سوالات پزشکی را جواب دهد، هم حقوقی و هم ورزشی؟

- روش غلط: استفاده از یک مدل غول‌پیکر برای همه کارها (هزینه بالا، کندی).
- روش صحیح: استفاده از معماری **Routing** (مسیریابی)

ساختار سیستم **Routing**:

۱. لایه اول: **(The Router)** یک مدل بسیار کوچک و سریع (مثلاً ۱.۷ میلیارد پارامتری یا حتی طبقه‌بندی‌کننده سنتی (BERT).

○ وظیفه: تشخیص نوع سوال. (Classification)

- Medical = روتر تشخیص می‌دهد کلاس -> "کاربر می‌گوید "سرم درد می‌کند: مثال"

۲. لایه دوم: **(Specialized Models)**

- (Fine-tuned for Medicine) Llama-۳ ارسال به مدل -> Medical اگر
- (Fine-tuned for Law) Mistral ارسال به مدل -> Legal اگر
- Gemma ارسال به مدل -> Sport اگر
- (General) ارسال به مدل عمومی -> Unknown اگر

مزیت این معماری: شما با هزینه بسیار کمتر، دقت بسیار بالاتری می‌گیرید چون هر سوال توسط متخصص همان حوزه پاسخ داده می‌شود.

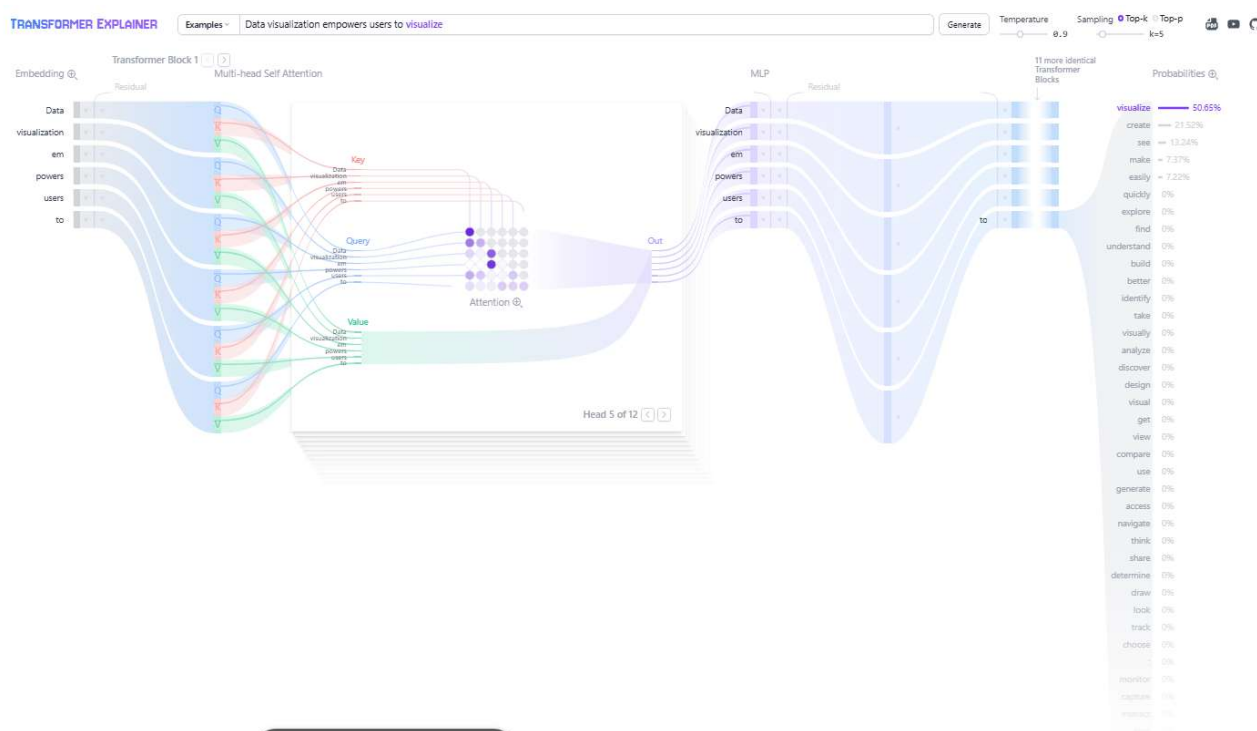
سناریوی پیشنهادی پروژه

به جای تلاش برای ساخت یک "چت بات همه کاره" با یک مدل سنگین، روی ساختن ایجنت های تخصصی کوچک تمرکز کنید که توسط یک Router مرکزی مدیریت می شوند. این معماری مقیاس پذیر (Scalable) و اقتصادی است.

فصل ۱۲: مهندسی پرامپت و ساختار پیام ها

۱. ابزار بصری سازی ترنسفورمر (Transformer Explainer)

یکی از دانشجویان (هادی) لینکی به ابزار Transformer Explainer ارسال کرد.



- اهمیت: این ابزار به صورت گرافیکی نشان می دهد که توکن ها چگونه وارد مدل می شوند، در لایه های Attention چه اتفاقی می افتد و چگونه خروجی تولید می شود.
- توصیه مدرس: حتماً این ابزار را بررسی کنید. درک اینکه "۱۲ هدِ اتنشن (Attention Heads)" چگونه همزمان روی متن کار می کنند، دید شما را باز می کند.

۲. ساختار پیام ها در LLM (Message Roles)

وقتی با مدل زبانی صحبت می کنیم، همه چیز متن ساده نیست. هر پیام یک "نقش (Role)" دارد که به مدل می فهماند چطور با آن برخورد کند.

الف) نقش کاربر (User Role)

- **تعریف:** همان پیامی است که کاربر نهایی تایپ می کند (مثلاً "یه ایمیل بنویس").
- **کارکرد:** مدل این را به عنوان "دستور جدید" می بیند که باید اجرا شود.

(ب) نقش سیستم (System Role)

- **تعریف:** مهم ترین بخش برای توسعه دهنده. این پیام **شخصیت، محدودیت ها و قوانین** مدل را تعیین می کند. کاربر نهایی معمولاً این پیام را نمی بیند.
- **جایگاه:** قبل از پیام کاربر به مدل ارسال می شود.
- **مثال:**

"تو یک دستیار نگارش نامه های اداری هستی. لحن تو باید رسمی و محترمانه باشد. پاسخ ها نباید بیشتر از ۲۰۰ کلمه باشند".

۳. دمو عملی: ربات اصلاح کننده ایمیل (Email Optimizer)

مدرس یک سناریوی واقعی را در محیط **n8n** پیاده سازی کرد.

سناریو:

کاربر یک متن محاوره و شلخته به ربات تلگرام می فرستد و ربات آن را به یک نامه اداری شیک تبدیل می کند.

اجزای Workflow :

۱. **Telegram Trigger:** دریافت پیام کاربر ("می خوام به مدیر تهران دیتا بگم کلاس رو زیاد کنه").
۲. **LLM Node (System Message):**
 - دستور: "تو به عنوان دستیار نگارش فعالیت می کنی. ورودی کاربر را به یک ایمیل رسمی، دقیق و ساختارمند تبدیل کن".
 - قید: "تعداد کلمات زیر ۲۰۰ باشد".
۳. **Code Node (JavaScript):** یک قطعه کد ساده برای اعمال محدودیت سخت افزاری (مثلاً چک کردن طول متن) یا فرمت دهی نهایی.
۴. **Telegram Response:** ارسال متن تمیز شده به کاربر.

نتیجه تست:

- ورودی کاربر: "من میخوام به مدیر بگم کلاس پنجشنبه رو زیاد کنه".
- خروجی مدل: "مدیر محترم مجموعه تهران دیتا، با سلام و احترام... خواهشمند است در صورت امکان تعداد جلسات مبحث ترنسفورمرها افزایش یابد".

۴. فلسفه پرامپت نویسی (Prompt Engineering)

جمله طلایی: "در دنیایی که همه جواب‌ها وجود دارد، برنده کسی است که سوال بهتری می‌پرسد".

- مدل‌های زبانی مثل ChatGPT جواب هر سوالی را دارند، اما کیفیت جواب مستقیماً به کیفیت سوال (Prompt) شما بستگی دارد.
- چالش: هیچ‌کس حوصله ندارد هر بار یک پرامپت طولانی و دقیق (مثل مثال بالا) بنویسد.
- راه حل: ما به عنوان توسعه‌دهنده، این پرامپت‌های طولانی را در لایه **System Message** پنهان می‌کنیم تا کاربر فقط با یک جمله ساده، بهترین خروجی را بگیرد. این همان کاری است که **Custom GPTs** یا ابزارهایی مثل **Jasper** انجام می‌دهند.

تمرین پیشنهادی

در پنل کاربری (LLM مثل ChatGPT یا Playground)، سعی کنید یک شخصیت بسازید. در بخش **System Instructions** بنویسید: "تو یک معلم سخت‌گیر ریاضی هستی که فقط با سوال جواب می‌دهی." سپس رفتار مدل را تست کنید.

فصل ۱۳: تکنیک‌های پیشرفته پرامپت و ابزار Dify

۱. تکنیک Prompt Generator (پرامپت‌ساز)

همانطور که گفتیم، نوشتن یک پرامپت کامل و دقیق (مثل مثال ایمیل) کار سخت و زمان‌بری است.

- مشکل: ما معمولاً حوصله یا دانش زبانی کافی برای نوشتن پرامپت‌های طولانی و حرفه‌ای (به خصوص به انگلیسی) را نداریم.
- راه حل هوشمندانه: استفاده از خود LLM برای ساخت پرامپت!
 - روش کار: به ChatGPT بگویید: "تو الان یک Prompt Engineer هستی. من می‌خواهم یک ایمیل رسمی برای مدیر مالی بنویسم. به من کمک کن و یک پرامپت دقیق و حرفه‌ای برای این کار بنویس."
 - وظایف اصلی Prompt Engineer:
 - طراحی پرامپت‌های هوشمند برای گرفتن خروجی درست از مدل‌های AI.
 - آزمایش و بهبود مداوم پرامپت‌ها برای افزایش کیفیت پاسخ‌ها.
 - تبدیل نیاز کسب‌وکار یا کاربر به پرامپت قابل فهم برای مدل.
 - مستندسازی بهترین پرامپت‌ها و ساخت «کتابخانه پرامپت».
 - همکاری با تیم‌های فنی و غیر فنی برای استفاده بهتر از AI.
 - نتیجه: مدل یک پرامپت عالی تولید می‌کند. شما این خروجی را به عنوان ورودی نهایی به مدل اصلی می‌دهید.

کاربرد: این روش به خصوص در تولید تصویر (Image Generation) بسیار حیاتی است. نوشتن توصیفات بصری دقیق برای Midjourney سخت است، اما اگر از ChatGPT بخواهید "توصیف یک غروب در سبک سایبرپانک" را بنویسد، نتیجه فوق العاده خواهد بود.

۲. استفاده از صوت به جای متن (Voice Typing)

یکی از راه‌های بهبود ورودی، استفاده از صوت است.

- چرا؟ چون حرف زدن راحت‌تر از تایپ کردن است. وقتی تایپ می‌کنید، معمولاً خلاصه می‌نویسید (۲ خط). اما وقتی حرف می‌زنید، راحت ۲۰ خط توضیح می‌دهید.
- مزیت: هرچه ورودی (Context) بیشتر باشد، خروجی مدل دقیق‌تر است.
- تکنولوژی: استفاده از مدل‌های (ASR (Automatic Speech Recognition مثل سایت Typo شما حرف می‌زنید، متن تبدیل می‌شود و به عنوان یک پرامپت غنی (Rich Prompt) به LLM داده می‌شود.

۳. معرفی پلتفرم (Dify دیفای)

مدرس ابزاری ایرانی به نام **Dify.ai** را معرفی کرد که برای ساخت سریع چت‌بات‌ها و POC (Proof of Concept) بسیار مفید است.

ویژگی‌های کلیدی:

- **No-Code/Low-Code**: امکان ساخت چت‌بات بدون کدنویسی پیچیده.
- **اتصال به RAG**: قابلیت آپلود فایل (مثلاً PDF قوانین شهرداری) و پرسش و پاسخ از روی آن.
- **مثال عملی (شهرداری تهران):**
 - آپلود ۷۰-۸۰ صفحه قوانین "ماده ۱۰۰" و تراکم ساختمانی.
 - کاربر (صوتی) می‌پرسد: "من یه زمین ۱۸۰ متری خریدم، چند تا پارکینگ می‌تونم داشته باشم؟"
 - سیستم تبدیل صوت به متن می‌کند -> در اسناد جستجو می‌کند -> (RAG) جواب دقیق می‌دهد.
- **خروجی**: امکان دریافت خروجی به صورت iframe برای قرار دادن در سایت یا استفاده به عنوان وب‌اپلیکیشن مستقل.
- **نکته**: این ابزار برای دمو و MVP عالی است، اما برای پروژه‌های Enterprise با هزاران کاربر همزمان، شاید نیاز به بررسی بیشتر داشته باشد) یا استفاده از API مستقیم.

۴. جمع‌بندی مسیر تا اینجا

تا این لحظه چه یاد گرفتیم؟

۱. مبانی: LLM چیست و چگونه کار می کند. (Transformer, Token)

۲. معماری: تفاوت SLM و LLM، مفهوم Embedding.

۳. تعامل: ساختار پیام ها (User/System Role) و مهندسی پرامپت.

۴. ابزارها: n^n برای اتوماسیون، Dify.ai برای ساخت سریع چت بات.

قدم بعدی: حالا نوبت ورود به دنیای واقعی کدنویسی و استفاده از API ها در سطح حرفه ای است.

فصل ۱۴: پیاده سازی عملی؛ از کال سنتر هوشمند تا API

۱. مقایسه ابزارها (Dify vs Flowise)

- (Dify دیفای اصل) : یکی از محبوب ترین پلتفرم های متن باز برای ساخت چت بات .
 - مزایا: کامیونیتی بزرگ (۴۵ هزار ستاره گیت هاب)، اتصال راحت به تلگرام و گوگل، رابط کاربری تمیز.
 - مجوز: Apache License (License): بسیار آزاد و مناسب برای استفاده تجاری).
- (Dify نسخه فارسی شده): (نسخه ای که مدرس برای استفاده داخلی شرکت فارسی سازی و شخصی سازی کرده است (چون نسخه اصلی فارسی ندارد).
- Flowise: ابزاری دیگر که مبتنی بر نودهاست (Node-based) اما به گفته مدرس، Dify برای شروع و رسیدن سریع به محصول نهایی (MVP) بهتر و کامل تر است.

ویژگی	Dify	Flowise	NAN
حداقل CPU	۲ هسته	۲ هسته	۱ هسته
حداقل RAM	۴ گیگابایت	۲ گیگابایت	۲ گیگابایت
فناوری فرانت اند	React/TypeScript	React/TypeScript	Vue.js
پشتیبانی از فارسی/RTL	بله	محدود	محدود
لایسنس	۲,۰ Apache با محدودیت	۲,۰ Apache	Sustainable Use License
ستاره های GitHub	۱۲۰ هزار	۴۵.۴ هزار	۱۶۰ هزار
تعداد مشارکت کنندگان	+۲۰۰	۱۹۲	+۴۵۰
قابلیت شاخص	Workflow LLM, RAG	LangChain visual builder	اتصال ۴۰۰+ سرویس

۲. دمو: هوشمندسازی بیمارستان (API Integration)

چگونه می‌توان هوش مصنوعی را به سیستم نوبت‌دهی بیمارستان متصل کرد؟

- سناریو: بیمار می‌پرسد: "من باردارم، دوشنبه صبح کدوم دکتر هست؟"
 - فرآیند پشت صحنه:
 ۱. AI درخواست را می‌فهمد
-> Intent: Find_Doctor, Specialty: OB/GYN, Time: Monday Morning.
 ۲. API Call: هوش مصنوعی یک درخواست HTTP مثل GET یا POST به سرور بیمارستان می‌فرستد.
 ۳. Response: سرور بیمارستان لیست پزشکان را به صورت JSON برمی‌گرداند.
 ۴. AI: لیست خام را به متن محاوره‌ای تبدیل می‌کند: "برای دوشنبه صبح، دکتر صدف در کلینیک حضور دارند."
- نکته فنی: در Dify می‌توانید نودهای HTTP Request داشته باشید که دقیقاً مثل Postman عمل می‌کنند و دیتای واقعی (Real-time) را به چت‌بات تزریق می‌کنند.
-

۳. دمو: کال سنتر هوشمند بانکی (VoIP + AI)

مدرس یک دمو زنده از تماس تلفنی با هوش مصنوعی (بانک گردشگری) اجرا کرد.

معماری سیستم:

۱. تلفن (VoIP): کاربر زنگ می‌زند و صحبت می‌کند.
۲. ASR (Speech-to-Text): صدای کاربر ("وام ازدواج می‌خوام") به متن تبدیل می‌شود.
۳. RAG (Retrieval): سیستم در داکيومنت‌های بانک (شرایط وام) جستجو می‌کند.
۴. LLM: جواب را می‌سازد ("وام ازدواج برای افراد زیر ۵۰ سال ۳۵۰ میلیون تومان است...").
۵. TTS (Text-to-Speech): متن جواب به صدای طبیعی تبدیل و برای کاربر پخش می‌شود.

نکته جالب دمو:

- کاربر پرسید: "رفیقم ۶۵ سالشه می‌خواه زن بگیره، وام میدین؟"
 - هوش مصنوعی (با درک منطق): "وام ازدواج برای بالای ۵۰ سال صادر نمی‌شود، بنابراین برای دوست شما نمی‌توانیم وام بدهیم."
 - نتیجه: مدل نه تنها جستجو کرد، بلکه استدلال (Reasoning) هم انجام داد و شرایط سنی را با سن دوست کاربر تطبیق داد.
-

۴. کنترل رفتار با System Message

چگونه جلوی پاسخ‌های نامربوط را بگیریم؟ (مثلاً اگر وسط تماس بانکی دستور پخت قورمه‌سبزی بخواهند!)

• System Prompt:

"تو یک دستیار هوشمند بانکی هستی. فقط به سوالات مربوط به وام و خدمات بانکی پاسخ می‌دهی. اگر سوالی غیر از این پرسیده شد، مودبانه بگو: در حیطه تخصص من نیست."

- این همان جایی است که ما API و OpenAI Compatible Endpoints را تنظیم می‌کنیم تا مدل دقیقاً طبق سناریوی ما رفتار کند.

چالش اصلی

بزرگترین چالش در این سیستم‌ها تاخیر (Latency) است. کاربر پشت تلفن نمی‌تواند ۱۰ ثانیه منتظر بماند. تمام هنر مهندس هوش مصنوعی این است که این پایپ‌لاین (ASR -> LLM -> TTS) را در کمتر از ۲-۱ ثانیه اجرا کند.

فصل ۱۵: مدل‌های بینایی (VLM)، پروتکل MCP و اتوماسیون

۱. مدل‌های بینایی (Vision Language Models - VLM)

LLMها فقط محدود به متن نیستند. مدل‌های جدید (مثل GPT-۴o یا Llama Vision) می‌توانند تصاویر را هم "ببینند" و تحلیل کنند.

دمو: تحلیل چک صیادی (بانک ملت)

مدرس یک سناریوی واقعی بانکی را توضیح داد:

- ورودی: تصویر یک چک بانکی.
- پردازش: مدل VLM تصویر را اسکن می‌کند و اطلاعات را استخراج می‌کند.
- خروجی:
 - نام صادرکننده: "علی نقی شوشتریان."
 - مبلغ به حروف: "..."
 - مبلغ به عدد: "..."
 - کشف خطا: مدل متوجه شد که مبلغ عددی با مبلغ حروفی ۱۰ برابر اختلاف دارد و هشدار داد: "اشتباه در مبلغ چک."

کاربرد: این یعنی هوش مصنوعی می‌تواند جایگزین متصدی بانک در بررسی اولیه اسناد شود.

۲. پروتکل MCP (Model Context Protocol)

یک مفهوم جدید و انقلابی (معرفی شده توسط Anthropic) برای اتصال LLM به دنیای بیرون.

- **مشکل فعلی:** اتصال LLM به دیتابیس یا CRM نیاز به نوشتن کدهای پیچیده و API های اختصاصی برای هر ابزار دارد.
- **راه حل MCP:** یک استاندارد واحد (Standard Protocol) که اجازه می دهد مدل های زبانی به راحتی به هر منبع داده ای (Google Drive, Slack, GitHub, Database) وصل شوند.
- **سناریو:** "من نیاز به محاسبه دارم؟ MCP -> "مدل را به ماشین حساب وصل می کند. "نیاز به دیتای مشتری دارم؟ -> "MCP مدل را به CRM وصل می کند.

۳. نصب و راه اندازی ابزارها (Docker & n^n)

برای پیاده سازی این سیستم ها در محیط شخصی یا سازمانی، ما از Docker و ابزار اتوماسیون n^n استفاده می کنیم.

نصب با داکر (Docker)

- چرا داکر؟ چون نصب دستی ابزارها (Dify, n^n, Flowise) با تمام وابستگی هایشان (Python, Node.js, Databases) کابوس است. داکر همه چیز را در یک کانتینر ایزوله تحویل می دهد.
 - **روش یادگیری:** مدرس پیشنهاد کرد به جای حفظ کردن دستورات، از ChatGPT کمک بگیرید:
- "من می خواهم n^n را با داکر روی سرور اوبونتو نصب کنم. مرحله به مرحله دستورات را به من بده و بگو هر دستور چه کار می کند."

اتوماسیون با n^n (شبیه Make ولی رایگان و Self-hosted)

- **اشتراک گذاری Workflow:** کل منطق برنامه شما در n^n یک فایل JSON است. می توانید این فایل را کپی کنید (Ctrl+C) و برای همکارانتان بفرستید (Ctrl+V) تا عیناً همان سیستم را داشته باشند.
- **استفاده از تمپلیت ها:** سایت n^n هزاران تمپلیت آماده دارد.
 - مثال: "چت بات تلگرام متصل به Google Sheets"
 - کافیست تمپلیت را کپی کنید، API Key خود را وارد کنید و تمام!

۴. چالش های زیرساختی در ایران

- **تحریم داکر:** متأسفانه docker pull در ایران فیلتر/تحریم است.
- **راه حل:** استفاده از ابزارهایی مثل شکن (Shecan) یا تنظیم پروکسی روی داکر. (docker proxy settings)

نکته مهم برای سازمان‌ها

ابزارهایی مثل Dify یا nⁿ برای ساخت سریع محصول (MVP) و تست ایده عالی هستند. اما برای مقیاس‌های بزرگ (Enterprise)، ممکن است نیاز به توسعه سیستم اختصاصی داشته باشید. هرچند بسیاری از شرکت‌ها هم‌اکنون با همین ابزارها سرویس‌های پایدار ارائه می‌دهند.

فصل ۱۶: پرسش و پاسخ زیرساختی و آمادگی برای فاز عملی

۱. پاسخ به سوالات فنی دانشجویان

سوال: آیا می‌توانم همزمان چندین مدل (متن، تصویر، صدا) را اجرا کنم؟

- پاسخ: بله، اما محدودیت اصلی شما **VRAM** (حافظه گرافیکی) است.
- توضیح: هر مدل هوش مصنوعی برای اجرا شدن باید روی **VRAM** کارت گرافیک بارگذاری شود.
 - اگر ۳ مدل داشته باشید (مثلاً **Llama** برای متن، **Whisper** برای صدا، **Stable Diffusion** برای تصویر)، هر کدام بخشی از رم کارت گرافیک را اشغال می‌کنند.
 - اگر یک کارت گرافیک **A40** (با ۴۸ گیگابایت حافظه) دارید، می‌توانید هر سه را همزمان بالا بیاورید. در غیر این صورت، سیستم کند شده یا کرش می‌کند.
 - راهکار مقیاس‌پذیری (**Scalability**): برای سرویس‌دهی به هزاران کاربر، باید چندین سرور و چندین کارت گرافیک داشته باشید. (**Load Balancing**)

سوال: چگونه مدل‌ها را ارزیابی کنیم (**Evaluate**)؟

- پاسخ: یکی از بهترین روش‌ها **LLM-as-a-Judge** است.
- توضیح: شما یک مدل کوچک (مثلاً مدل حقوقی فاین‌تیون شده) دارید. برای اینکه بفهمید درست جواب می‌دهد یا نه، خروجی آن را به یک مدل قدرتمندتر و هوشمندتر (مثل **GPT-4** یا **Gemini**) می‌دهید و می‌گویید: "آیا این جواب از نظر حقوقی درست است؟ به آن نمره بده".

سوال: سرور ویندوزی یا لینوکسی؟

- پاسخ: در دنیای هوش مصنوعی و سرور، **ویندوز جایی ندارد!** همه چیز روی **لینوکس** است.
- توصیه: نگران نباشید. نیازی نیست متخصص لینوکس باشید. کفایت یک سرور لینوکسی (**VPS**) اجاره کنید و با ابزارهایی مثل **Termius** یا **PuTTY** به آن وصل شوید. دستورات را هم می‌توانید از **ChatGPT** بپرسید ("دستور نصب داکر در اوبونتو چیست؟").

۲. معرفی ابزارها برای بخش بعدی

بعد از استراحت، وارد فاز کاملاً عملی و کدنویسی می‌شویم. ابزارهای مورد نیاز:

۱. **Google Colab**: محیط اجرای کد پایتون به صورت رایگان (نیاز به پروکسی/فیلترشکن دارد).
۲. **OpenRouter API**: برای دسترسی به مدل‌های زبانی.
۳. **Gradio**: کتابخانه‌ای برای ساخت سریع رابط کاربری (Web App) چت‌بات‌ها.

۳. خلاصه وضعیت کلاس

- تا اینجا مبانی تئوری، معماری مدل‌ها، پرامپت نویسی و ابزارهای No-Code (مثل n^n و Dify) را پوشش دادیم.
- بخش بعدی: کدنویسی **Python**، کار با **API** و ساخت **Web Application**.

زمان استراحت: کلاس برای ناهار/استراحت کوتاه متوقف می‌شود.

پایان ساعت دوم

مبحث بعدی: شروع کار با **API** ها و ساخت وب اپلیکیشن با **Gradio**

فصل ۱۷: منابع مطالعاتی و مقدمه‌ای بر API

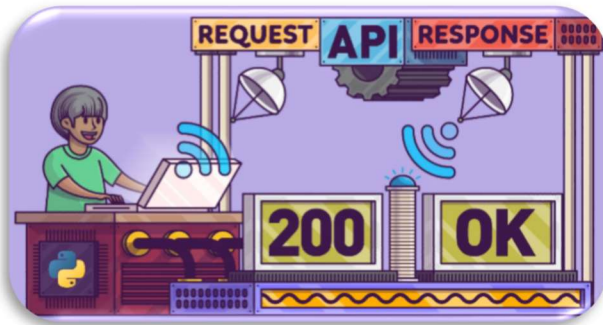
۱. تکلیف هفته آینده (۱۱۵ سوال LLM)

مدرس یک فایل PDF حاوی ۱۱۵ سوال کلیدی مصاحبه شغلی و مفاهیم **LLM** را در گروه تلگرام به اشتراک گذاشت.

- هدف: این سوالات هم مرور مباحث کلاس است و هم بنچمارکی برای سنجش سطح دانش شما.
- وظیفه: تا هفته آینده، حداقل ۵۰ سوال اول را مطالعه کنید.
 - مثال: سوال ۱۱۵ در مورد MoE (Mixture of Experts) است که بعداً توضیح داده خواهد شد.
 - اگر سوالی را بلد نیستید، نگران نباشید. هدف این است که نقاط ضعف خود را بشناسید تا یاد بگیرید.

لینک فایل: در گروه تلگرام سنجاق شده است (فایل پاورپوینت: اسلاید ۶۳ مگابایتی کلاس نیز آپلود شد).

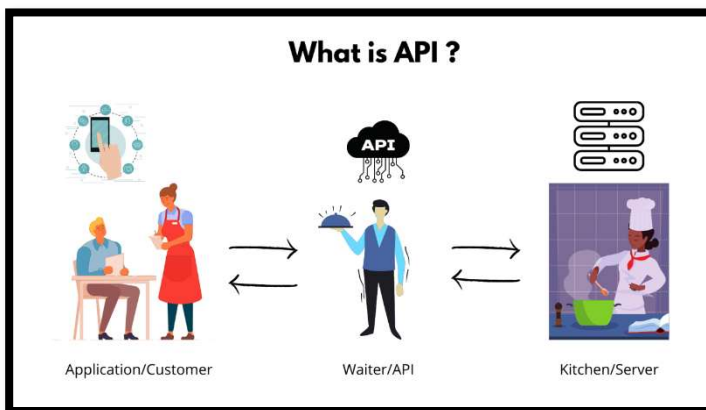
۲. چرا API مهم است؟



ما تا اینجا مفاهیم (Token, Temperature, Prompt) را یاد گرفتیم. اما در دنیای واقعی، ما با پنل گرافیکی ChatGPT کار نمی‌کنیم. ما کد می‌نویسیم و کدهای ما باید با مدل‌ها صحبت کنند. زبان مشترک بین کد ما و مدل‌ها، API است.

تعریف ساده API (Application Programming Interface)

API مثل یک گارسون در رستوران است.



- شما (Client): مشتری هستید که روی میز نشسته‌اید و غذا (سرویس) می‌خواهید.
- آشپزخانه (Server/LLM): جایی است که غذا آماده می‌شود (پردازش انجام می‌شود).
- API (گارسون): سفارش شما را می‌گیرد، به آشپزخانه می‌برد و غذا را برای شما می‌آورد. شما نیازی ندارید وارد آشپزخانه شوید و خودتان غذا بپزید.

مثال‌های روزمره:

- تاکسی اینترنتی (Snapp/Uber): وقتی شما مبدأ و مقصد می‌زنید، اپلیکیشن با API به سرور نقشه (Google Maps یا OSM) وصل می‌شود، ترافیک و مسافت را می‌گیرد و قیمت را به شما نشان می‌دهد.
- دیجی کالا: وقتی قیمت یک کالا را می‌بینید، مرورگر شما یک درخواست API به سرور دیجی کالا فرستاده و قیمت را گرفته است.

۳. معماری‌های ارتباطی

ما برای ارتباط با سرویس‌های هوش مصنوعی، عمدتاً با دو مفهوم سروکار داریم:

الف) REST (Representational State Transfer)

- این استانداردترین روش ارتباط در وب است.
- مبتنی بر پروتکل HTTP است (همان پروتکلی که وبسایت‌ها با آن باز می‌شوند).
- فرمت داده: معمولاً JSON است (همان ساختار {Key: Value} که در پایتون دیکشنری نام دارد).
- کاربرد ما: ۹۹٪ کار ما با OpenAI, Anthropic, OpenRouter از طریق REST API است.

Web Socket(ب)

- برای ارتباط **Real-time** (لحظه‌ای) و دوطرفه استفاده می‌شود.
- مثال: چت پشتیبانی آنلاین، قیمت لحظه‌ای بورس، لوکیشن زنده تاکسی.
- در این روش، کانال ارتباطی باز می‌ماند تا سرور هر وقت خواست به کلاینت پیام بدهد (مثل Push Notification).

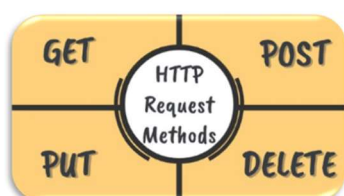
جمع‌بندی: ما باید یاد بگیریم چطور با پایتون یک "Request" (سفارش) به فرمت JSON بسازیم، آن را به "API" بدهیم و "Response" (غذا) را تحویل بگیریم.

نقشه راه ساعت سوم

در این ساعت، ما دقیقاً یاد می‌گیریم:

۱. چطور یک API Request بسازیم. (POST, GET)
۲. چطور با سرویس **OpenRouter** کار کنیم.
۳. چطور این‌ها را در کد پایتون پیاده‌سازی کنیم.
۴. و در نهایت، چطور با کتابخانه **Gradio** یک وب‌اپلیکیشن چت‌بات بسازیم.

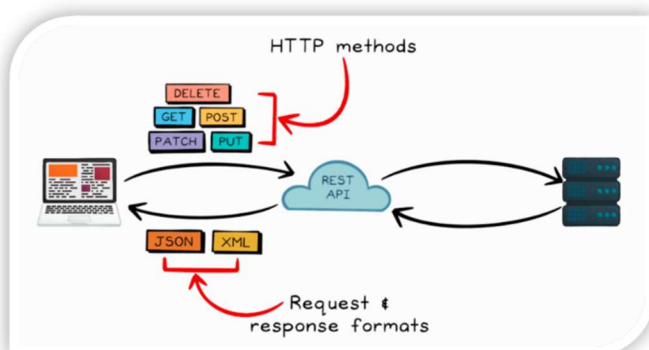
فصل ۱۸: متدهای HTTP و کدهای وضعیت



:REST API

رایج‌ترین نوع API که بر پایه‌ی HTTP کار می‌کند.

درخواست‌ها (Request) با متدهایی مثل GET, POST, PUT, DELETE ارسال می‌شوند.



:متدهای اصلی HTTP

- GET: دریافت داده (مثلاً گرفتن لیست کاربران).
- POST: ارسال داده جدید (مثلاً ایجاد کاربر).
- PUT: بروزرسانی کامل داده (مثلاً تغییر مشخصات یک کاربر).
- PATCH: بروزرسانی بخشی از داده.
- DELETE: حذف داده.

۱. متدهای اصلی (GET vs POST) HTTP

وقتی با API صحبت می‌کنیم، باید نوع درخواستمان را مشخص کنیم. دو متد اصلی که در هوش مصنوعی استفاده می‌کنیم عبارتند از:

الف) متد GET (فقط خواندن)

- **تعریف:** مثل نگاه کردن به منوی رستوران است. شما فقط اطلاعات را می‌گیرید و چیزی را تغییر نمی‌دهید.
- **کاربرد:**
 - پرسیدن لیست مدل‌های موجود از OpenRouter (Get Models List).
 - چک کردن قیمت دلار.
 - مشاهده یک وبسایت.
- **مثال:** لیست تمام مدل‌هایی که داری را به من بده."

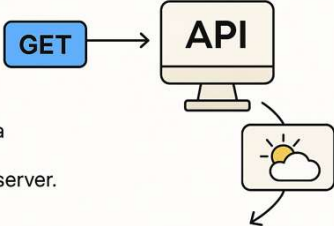
GET

Description
GET is used to retrieve data from a server.
No data is modified on the server.

Example

```
https://api.weatherapi.com/v1/current.json?key=API_KEY?q=Tehran
```

```
import requests
url = https://apiweatherapi.com/v1/current.json.get()
print(response.get('ysl.g'))
```



متد GET

- GET برای دریافت داده از سرور استفاده می‌شود.
- هیچ تغییری در داده‌های سرور ایجاد نمی‌کند (فقط خواندن).
- پارامترها می‌توانند از طریق Query String در URL فرستاده شوند.

ب) متد POST (ارسال و تغییر داده)

- **تعریف:** مثل سفارش دادن غذاست. شما اطلاعات جدیدی (سفارش) را به سرور می‌فرستید تا کاری برایتان انجام دهد.
- **کاربرد:**
 - ارسال پرامپت به ChatGPT برای گرفتن جواب.
 - ارسال تصویر چک به مدل بینایی.
 - ثبت نام در سایت یا ارسال کامنت.
- **نکته مهم:** در متد POST، ما معمولاً یک بسته داده (Payload) به فرمت JSON همراه درخواست می‌فرستیم که حاوی اطلاعات ماست (مثلاً متن پرامپت).

POST

متد POST `https://jsonplaceholder.typicode.com/posts`

بدنه

```
{
  "customer": "Ali",
  "product": "Laptop",
  "quantity": 2,
  "price": 1500
}
```

متد POST

- POST برای ارسال داده جدید به سرور استفاده می‌شود.
- منجر به ایجاد داده یا شروع یک پردازش جدید در سرور می‌شود. داده‌ها معمولاً در بدنه (Body) درخواست ارسال می‌شوند (به شکل JSON، XML یا فرم).
- پاسخ سرور معمولاً شامل وضعیت ایجاد موفق مثل 201 (Created) و گاهی شناسه (ID) داده جدید است.

۲. کدهای وضعیت (HTTP Status Codes)

وقتی درخواستی می‌فرستید، سرور علاوه بر جواب اصلی، یک "کد وضعیت" هم برمی‌گرداند که به شما می‌گوید اوضاع چگونه است. این کدها برای عیب‌یابی (Debugging) حیاتی هستند.

رنگ کد	معنی کلی	مثال‌های رایج
۲xx	موفقیت (Success)	۲۰۰ OK: همه چیز عالی است (برای GET). ۲۰۱ Created: داده با موفقیت ثبت شد (برای POST).
۳xx	ریدایرکت (Redirection)	۳۰۱ Moved: آدرس تغییر کرده است.
۴xx	خطای کلاینت (Client Error)	۴۰۰ Bad Request: درخواست شما مشکل دارد (فرمت غلط). ۴۰۱ Unauthorized: رمز عبور یا API Key اشتباه است. ۴۰۴ Not Found: آدرس اشتباه است.
۵xx	خطای سرور (Server Error)	۵۰۰ Internal Server Error: سرور خراب است یا مشکلی دارد (تقصیر شما نیست). ۵۰۳ Service Unavailable: سرور شلوغ یا در حال تعمیر است.

قانون کلی:

- اگر کد ۴۰۰ گرفتید، کد خودتان را چک کنید (غلط املایی، فرمت اشتباه، کلید نامعتبر).
- اگر کد ۵۰۰ گرفتید، صبر کنید و دوباره تلاش کنید (مشکل از سمت سرویس‌دهنده است).

API response code categories

- 1xx – Informational
- 2xx – Success
- 3xx – Redirection
- 4xx – Client Error
- 5xx – Server Error

1xx	2xx	3xx	4xx	5xx
The request was received but there isn't a response yet	The request was received successfully and the server can return the requested data	The requested resource is in another location, so a separate call is necessary	The request can't be completed due to an issue on the client's end	The request can't be completed due to an issue on the server's end

دسته‌های اصلی Status Code

- ۲۰۰ ☒ موفق
- ۲۰۱ ☐ ساخته شد

- ۴۰۰ ✕ درخواست اشتباه
- ۴۰۱ 🔄 نیاز به ورود
- ۴۰۳ 🚫 دسترسی ممنوع
- ۴۰۴ 🔍 پیدا نشد
- ۵۰۰ ✨ خطای سرور

۳. پیاده‌سازی در پایتون (کتابخانه requests)

برای استفاده از این مفاهیم در پایتون، از کتابخانه قدرتمند `requests` استفاده می‌کنیم.

```
import requests

# مثال متد GET
response = requests.get("https://api.example.com/models")
print(response.status_code) # خروجی: ۲۰۰

# مثال متد POST
payload = {"prompt": "Hello AI", "model": "gpt-۴"}
response = requests.post("https://api.example.com/chat", json=payload)
print(response.json())
```

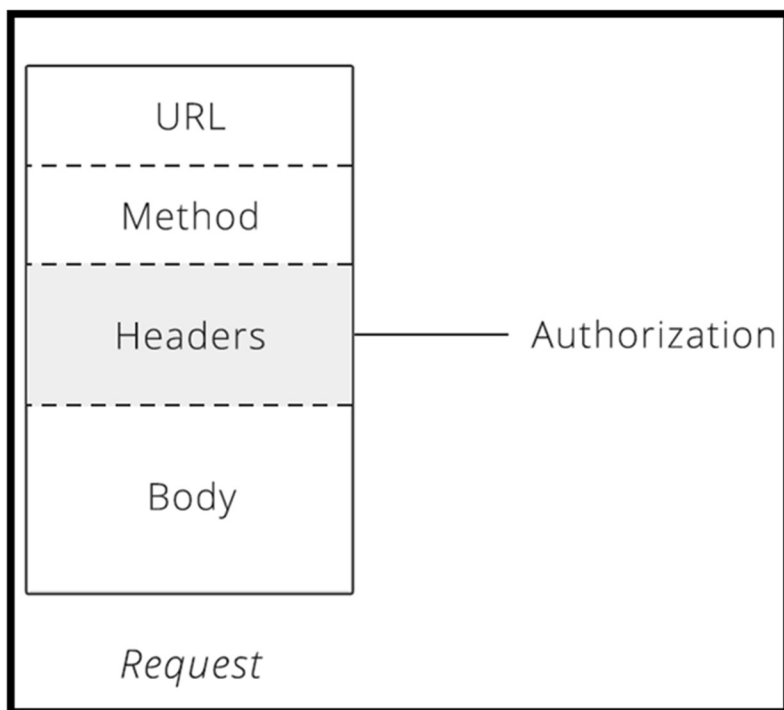
نکته: در بخش بعدی، دقیقاً همین کدها را برای اتصال به OpenRouter اجرا خواهیم کرد.

پایان مبانی تئوری API

فصل ۱۹: امنیت API، ابزار cURL و مفهوم Webhook

۱. احراز هویت در (API Authentication)

وقتی می‌خواهید از یک API استفاده کنید (مثلاً قیمت‌ها را بگیرید یا از مدل GPT-۴ استفاده کنید)، سرور باید بداند شما چه کسی هستید و آیا اجازه (یا اعتبار پولی) دارید یا نه؟



۴ روش اصلی احراز هویت:

۱. API Key (پرکاربردترین برای ما):

- سرویس‌دهنده (مثل OpenAI یا ElevenLabs) یک کد طولانی و محرمانه به شما می‌دهد.
- شما این کد را در **Header** درخواست خود قرار می‌دهید.
- مثال: `Authorization: Bearer sk-۱۲۳۴۵۶...`

۲. Basic Auth:

- استفاده از نام کاربری و رمز عبور ساده. (قدیمی شده و کمتر استفاده می‌شود).

۳. Bearer Token (JWT):

- مدرن‌ترین روش. سرور یک "توکن" موقت به شما می‌دهد که تا زمان انقضا (مثلاً ۱ ساعت) معتبر است.

۴. OAuth (مثل Login with Google):

- وقتی دکمه "ورود با گوگل" را می‌زنید، نیاز نیست پسوردتان را به سایت جدید بدهید. گوگل تایید می‌کند که شما، شما هستید. (برای لاگین کاربران استفاده می‌شود).

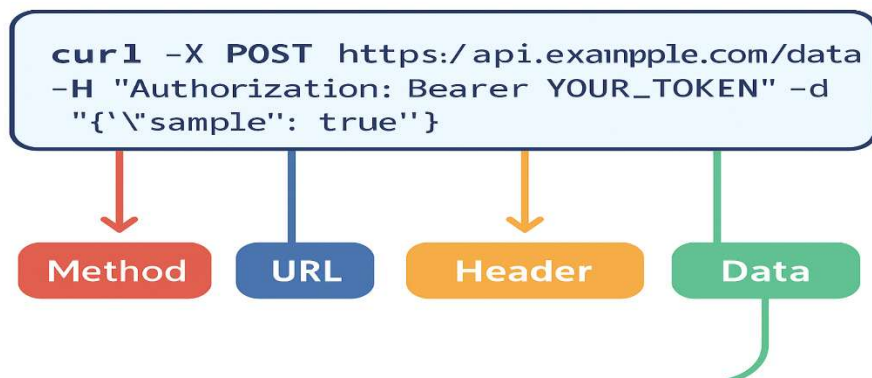
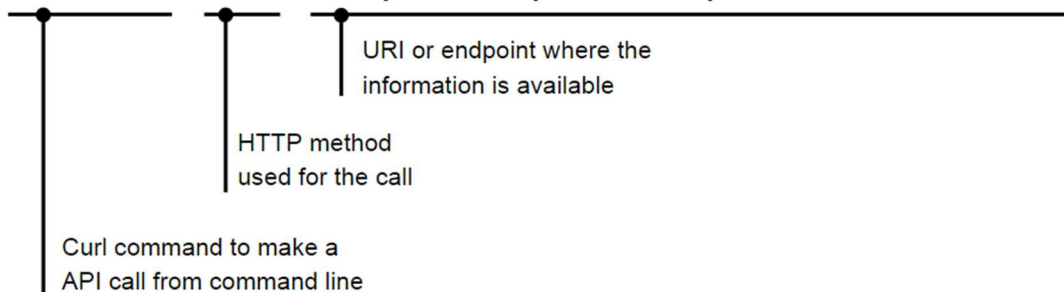
سوال دانشجو: فرق API Key و Bearer Token چیست؟ **پاسخ:** در عمل خیلی شبیه هستند. معمولاً API Key ثابت است (Static) و تغییر نمی‌کند، اما Token ممکن است موقتی باشد و بعد از مدتی منقضی شود. در بسیاری از API ها، ما API Key را با فرمت `Bearer sk-۱۲۳...` در هدر ارسال می‌کنیم.

۲. ابزار (cURL (Client URL

cURL یک ابزار خط فرمان (Command Line) است که در لینوکس و مک (و ویندوزهای جدید) وجود دارد. با cURL می‌توانید بدون نوشتن کد پایتون، یک درخواست API بفرستید.

- ساختار یک دستور cURL :

```
curl -X GET https://api.example.com/servers
```



- ```
curl -X POST "https://api.example.com/data" \
-H "Authorization: Bearer MY_API_KEY" \
-H "Content-Type: application/json" \
-d '{"key": "value"}'
```

○ -X: نوع متد. (GET/POST)

○ -H: هدرها (برای احراز هویت).

○ -d: دیتا. (Payload)

**کاربرد:** اکثر داکيومنت‌های API (مثل Stripe یا OpenAI) مثال‌هایشان را با cURL می‌نویسند. شما می‌توانید این دستور را کپی کنید و در ترمینال اجرا کنید تا سریع تست کنید.

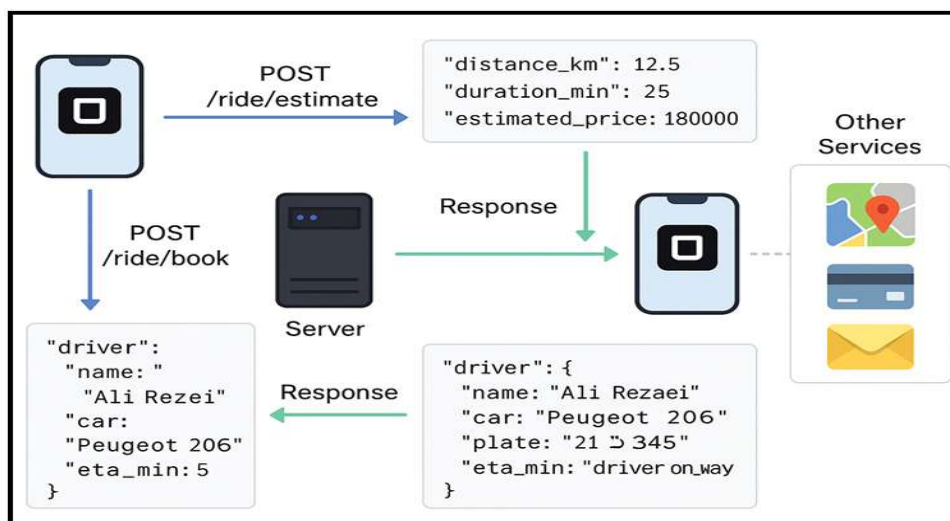
### مثال تاکسی اینترنتی

**Request:** آپ از سرور چیزی می‌خواه (مثل منو در رستوران)

**Response:** سرور جواب می‌ده (غذا حاضر میشه).

متدها: نوع عملیات (GET, POST, PUT, DELETE)

JSON: زبان مشترک برای رد و بدل کردن داده‌ها.



: اپ از سرور چیزی می‌خواهد (مثل منو در رستوران)

Response: سرور جواب می‌دهد (غذا حاضر میشه).

متدها: نوع عملیات (GET, POST, PUT, DELETE)

JSON: زبان مشترک برای رد و بدل کردن داده‌ها.

### ۳. مفهوم Webhook (تلفن اضطراری!)

#### • API معمولی (Polling):

- شما مدام به دوستان زنگ می‌زنید: "حالت خوبه؟" "نه". "۵ دقیقه بعد: "حالت خوبه؟" "نه".
- این کار منابع سرور را هدر می‌دهد و شما را خسته می‌کند.

#### • Webhook (Push):

- شما به دوستان می‌گویید: "هر وقت حالت بد شد، خودت به من زنگ بزن".
- شما منتظر می‌مانید (بدون ارسال درخواست اضافه) و به محض اینکه اتفاقی افتاد (Trigger)، سرور به شما خبر می‌دهد.

#### : کاربردهای واقعی Webhook

- اسنپ: وقتی راننده رسید، سرور به گوشی شما خبر می‌دهد. (Push Notification) شما مدام چک نمی‌کنید "رسید؟ رسید؟".
- بات تلگرام: وقتی کاربر پیامی می‌فرستد، تلگرام آن را به سرور شما "شلیک" می‌کند.
- پرداخت بانکی: وقتی کاربر پول را واریز کرد، درگاه پرداخت به سایت شما خبر می‌دهد "پرداخت موفق بود".

- **API**: ارتباط یک طرفه از سمت شما. (Client -> Server)
- **Webhook**: ارتباط یک طرفه از سمت سرور به شما (Server -> Client) بر اساس یک رخداد. (Event)

## آمادگی برای کدنویسی

مفاهیم تئوری تمام شد! در بخش بعدی، وارد **OpenRouter** می شویم، یک **API Key** واقعی می گیریم و اولین درخواست هوش مصنوعی خود را با پایتون ارسال می کنیم.

# فصل ۲۰: استاندارد OpenAI و ساختار پیامها

## ۱. مفهوم OpenAI Compatible API

- **استاندارد جهانی**: شرکت OpenAI یک ساختار خاص برای API خود تعریف کرد (نحوه ارسال پیام، دریافت جواب و...). این ساختار آنقدر محبوب شد که تبدیل به استاندارد صنعت شد.
- **مزیت بزرگ**: حالا شرکت های دیگر (مثل OpenRouter, Mistral, Ollama یا سرویس های ایرانی مثل AvalAI) هم دقیقاً از همان استاندارد استفاده می کنند.
- **نتیجه**: شما یک بار کد می نویسید (برای OpenAI)، اما می توانید با تغییر یک خط (URL یا Model Name)، مدل را عوض کنید. مثلاً به جای GPT-۴ از Llama ۳ استفاده کنید، بدون اینکه کدتان را تغییر دهید.

## ۲. نقش ها در پیام (Message Roles)

وقتی با مدل چت می کنیم، پیامها یک "برچسب" یا "نقش (Role)" دارند که مشخص می کند گوینده کیست. سه نقش اصلی داریم:

### ۱. نقش System (قوانین و چارچوب)

- **تعریف**: این پیام پنهان است و به مدل می گوید "تو کی هستی و چه کار باید بکنی."
- **کاربرد**: تعیین شخصیت و محدودیت ها. (Guardrails)
- **مثال**:

" . تو یک دستیار آشپزی هستی. اگر سوال غیر آشپزی پرسیدند، جواب نده"  
Role: system, Content:

### ۲. نقش User (کاربر)

- **تعریف**: پیامی که کاربر نهایی (من و شما) تایپ می کند.

- مثال:

Role: user, Content: "سرم درد می‌کنه، چی کار کنم؟"

### ۳. نقش Assistant (هوش مصنوعی)

- تعریف: جوابی که مدل قبلاً داده است.
- کاربرد حیاتی (حافظه): مدل‌های زبانی "حافظه" ندارند. برای اینکه چت را به یاد بیاورند، ما باید تاریخچه مکالمه (History) را دوباره برایشان بفرستیم.
- سناریوی حافظه:
  ۱. کاربر: "سرم درد می‌کنه".
  ۲. مدل: "قرص بخور." (این پیام حالا تبدیل به Role: assistant می‌شود)
  ۳. کاربر: "دندونم هم درد می‌کنه. به اون ربط داره؟"
  ۴. ارسال بعدی به مدل: ما کل این ۳ پیام را می‌فرستیم تا مدل بفهمد "اون" یعنی سردرد قبلی.

### ۳. پاسخ به سوالات ( Webhook و امنیت)

- Webhook در تلگرام:
  - به جای اینکه مدام از تلگرام بپرسید "پیام جدید اومد؟"، یک Webhook تنظیم می‌کنید. هر وقت کسی در گروه پیام داد، تلگرام آن پیام را به سرور شما "پرت" می‌کند تا پردازش کنید.
- امنیت LLM (Jailbreaking):
  - چطور جلوی کاربرانی که می‌خواهند مدل را فریب دهند (مثلاً ساخت بمب) بگیریم؟
  - این مبحث "Red Teaming" و ایمن‌سازی است که جلوتر بررسی خواهیم کرد. فعلاً با System Prompt جلوی بسیاری از این موارد را می‌گیریم.

### آماده برای عملیات

حالا که استاندارد پیام‌ها (System/User/Assistant) را فهمیدیم، همه چیز برای نوشتن کد پایتون آماده است. در بخش بعدی وارد Google Colab می‌شویم.

## فصل ۲۱: جمع‌بندی نهایی و تفاوت‌های امنیتی Key و Token

### ۱. چرا مدل‌های کوچک به System Prompt حساس‌ترند؟

- **مشکل Context Window:** مدل‌های کوچک (مثل ۷ یا ۱۴ میلیارد پارامتری) حافظه کوتاه‌تری دارند.
- **اهمیت System Role:** اگر شما یک پرامپت طولانی و پیچیده را به عنوان User بفرستید، ممکن است مدل گیج شود. اما اگر قوانین اصلی (شخصیت، محدودیت‌ها) را در System Role قرار دهید، مدل بهتر می‌فهمد که "چارچوب کلی من چیست" و پاسخ دقیق‌تری می‌دهد.
- **نکته:** در مدل‌های تجاری بزرگ (مثل ChatGPT و وبسایت)، خود شرکت یک System Prompt مخفی و بسیار قوی تنظیم کرده تا مدل وارد بحث‌های خطرناک نشود.

## ۲. جمع‌بندی نهایی: تفاوت API Key و Bearer Token

این سوال پرتکرار دانشجویان بود که مدرس آن را این‌گونه جمع‌بندی کرد:

| ویژگی         | API Key                                      | Bearer Token (JWT)                                     |
|---------------|----------------------------------------------|--------------------------------------------------------|
| ماهیت         | یک کلید ثابت و طولانی                        | یک توکن رمزنگاری شده و هوشمند                          |
| طول عمر       | معمولاً دائمی (تا وقتی خودتان لغو کنید)      | موقتی (مثلاً ۱ ساعت یا ۱ روز)                          |
| سطح دسترسی    | معمولاً دسترسی کامل به اکانت                 | قابلیت محدودسازی دقیق (Scopes)                         |
| سوالش از شما؟ | "رمز عبور شما چیست؟"                         | "شما کی هستید؟ چه اجازه‌ای دارید؟ تا کی اعتبار دارید؟" |
| امنیت         | اگر لو برود، خطرناک است (باید سریع باطل شود) | اگر لو برود، چون زود منقضی می‌شود، خطرش کمتر است       |

در این دوره: ما فعلاً با **API Key** کار می‌کنیم (چون ساده‌تر است)، اما در پروژه‌های حرفه‌ای و Enterprise، استفاده از Token استانداردتر است.

## ۳. خلاصه آنچه تا الان یاد گرفتیم (ساعت ۱ تا ۳)

ما از صبح تا الان مسیر زیر را طی کردیم:

۱. مبانی پایتون: دیکشنری‌ها (Dictionary) و ساختار JSON.
۲. معماری LLM: توکن‌ها، پرامپت‌نویسی، دما (Temperature).
۳. ابزارها: معرفی OpenRouter به عنوان هاب مدل‌ها.
۴. تئوری API: درخواست و پاسخ (Request/Response)، متدهای GET/POST و کدهای وضعیت.
۵. استاندارد OpenAI: نقش‌های System, User, Assistant.

## ۴. برنامه بعد از ناهار (ساعت ۱۴:۱۵ به بعد)

وارد فاز عملیاتی و کدنویسی می‌شویم (Zero to Hero):

۱. **Google Colab**: باز کردن نوت‌بوک و اجرای کدها.
۲. **API Call**: ارسال اولین درخواست واقعی به OpenRouter با پایتون.
۳. **Web App**: ساخت یک چت‌بات گرافیکی با **Gradio**.
۴. **اتصال**: وصل کردن بک‌اند (API) به فرانت‌اند (Gradio).

وضعیت کلاس: استراحت برای ناهار. ساعت ۱۴:۱۵ شروع مجدد.

### پایان ساعت سوم

فاز تئوری به پایان رسید .  
بخش بعدی: شروع کدنویسی پایتون و ساخت پروژه عملی

## فصل ۲۲: شروع کار عملی با OpenAI API و پایتون

### ۱. آماده‌سازی محیط Google Colab

در شروع بخش عملی، مدرس یک فایل نوت‌بوک (AI\_API.ipynb) را در اختیار دانشجویان قرار داد تا در Google Colab آپلود و اجرا کنند.

- **نصب کتابخانه**: اولین قدم، نصب پکیج رسمی OpenAI در پایتون است:
- `!pip install openai`

### ۲. دریافت API Key و مدیریت هزینه

مدرس وارد پنل کاربری (platform.openai.com) OpenAI شد و مراحل زیر را توضیح داد:

۱. **ساخت کلید**: در بخش API Keys، یک کلید جدید (sk-...) ایجاد کرد.
  - نکته/منیتی: این کلید فقط یک بار نمایش داده می‌شود. باید آن را کپی و در جایی امن ذخیره کنید.
۲. **بررسی اعتبار (Billing)**: در بخش Usage و Billing، میزان اعتبار باقیمانده (مثلاً ۳۲ دلار) و مصرف روزانه قابل مشاهده است.
۳. **محاسبه هزینه**:
  - سایت LLMTOKENCounter معرفی شد تا تعداد توکن‌های ورودی و خروجی محاسبه شود.

- مثال هزینه: اگر ورودی ۱۲ توکن و خروجی ۱۶۲ توکن باشد، هزینه بر اساس نرخ مدل (مثلاً) GPT-4o محاسبه می‌شود.
- قیمت: GPT-4o ورودی ۱.۷۵ دلار و خروجی ۱۴ دلار (به ازای هر یک میلیون توکن).

## ۳. اولین کد اتصال به هوش مصنوعی

مدرس یک کد پایه پایتون برای ارسال درخواست به OpenAI نوشت:

```
from openai import OpenAI

۱. # ساخت کلاینت با کلید API

client = OpenAI(api_key="sk-proj-...")

۲. # ارسال درخواست (Chat Completion)

response = client.chat.completions.create(
 model="gpt-۳,۵-turbo",
 messages=[
 {"role": "system", "content": "You are a helpful Persian assistant."},
 {"role": "user", "content": "قورمه‌سبزی چیست؟"}
],
 max_tokens=۵۰ # محدود کردن پاسخ برای کاهش هزینه
)

۳. # چاپ خروجی

print(response.choices[۰].message.content)
```

- توضیح کد: ما یک client ساختیم، مدل را gpt-۳,۵-turbo انتخاب کردیم و نقش System را "دستیار فارسی" تعریف کردیم.

## ۴. مفهوم استریمینگ (Streaming)

یکی از ویژگی‌های مهم تجربه کاربری (UX) در هوش مصنوعی، نمایش کلمه به کلمه پاسخ است (مثل تایپ کردن ChatGPT).

- حالت **stream=False** (پیش فرض):
  - برنامه صبر می‌کند تا کل پاسخ تولید شود، سپس همه را یکجا نشان می‌دهد.
  - مشکل: اگر پاسخ طولانی باشد، کاربر فکر می‌کند برنامه هنگ کرده است.
- حالت **stream=True**:
  - پاسخ به صورت تکه تکه (Chunk) ارسال می‌شود.
  - کد:

```
stream = client.chat.completions.create(
 model="gpt-۳,۵-turbo",
 messages=[{"role": "user", "content": "قورمه‌سبزی چیست؟"}],
 stream=True
)
for chunk in stream:
 if chunk.choices[۰].delta.content is not None:
 print(chunk.choices[۰].delta.content, end="")
```



- نتیجه: کاربر بلافاصله شروع تایپ شدن جواب را می بیند (حس زنده بودن).

## نکته مهم برای دانشجویان

چون دسترسی مستقیم به OpenAI از ایران (و بدون کارت اعتباری) دشوار است، در ادامه کلاس از سرویس واسطه OpenRouter استفاده خواهیم کرد که دقیقاً همین کدها را اجرا می کند، اما دسترسی به مدل های رایگان و ارزان را فراهم می کند.

## فصل ۲۳: ساخت چت بات با حافظه (Chat History)

### ۱. کدنویسی چت بات هوشمند OpenAI : (Part ۱) AI-API #

مدرس کدی نوشت که تا زمان تایپ کلمه exit، مکالمه را ادامه می دهد و تمام پیام های قبلی را به مدل یادآوری می کند.

#### منطق برنامه:

۱. یک لیست خالی messages ایجاد می کنیم.
۲. یک حلقه بی نهایت (while True) شروع می کنیم.
۳. پیام کاربر را می گیریم و با نقش user به لیست اضافه می کنیم.
۴. کل لیست (شامل پیام های قبلی) را به API می فرستیم.
۵. پاسخ مدل را می گیریم و با نقش assistant به لیست اضافه می کنیم.
۶. این چرخه تکرار می شود تا "حافظه" شکل بگیرد.

### ۲. تست عملی حافظه

- دور اول:
  - کاربر: "می خوام آش درست کنم، چی بخرم؟"
  - مدل: لیست حبوبات و سبزی آش را می دهد.
- دور دوم:
  - کاربر: "پیتزا چطور؟"
  - مدل: "برای پیتزا به خمیر، پنیر، سوسیس و... نیاز داری."
  - تحلیل: اگر حافظه نبود، مدل نمی پرسید "پیتزا چطور چی؟". چون تاریخچه قبلی ("چی بخرم") را داشت، فهمید منظور کاربر "مواد لازم برای پیتزا" است.

### ۳. چالش بزرگ: هزینه و محدودیت پنجره (Context Window)

مدرس سوال مهمی پرسید: "این روش چه مشکلی دارد؟" پاسخ دانشجویان (محمد و جابر) درست بود: افزایش تصاعدی توکن‌ها.

## تحلیل مشکل:

### ۱. افزایش هزینه:

- دور اول: ۱۰ توکن ورودی.
- دور دهم: ۱۰۰۰ توکن ورودی (چون ۱۰ پیام قبلی دوباره ارسال می‌شوند).
- شما برای پیامی که ۱۰ دقیقه پیش دادید، در هر بار چت جدید، دوباره پول پرداخت می‌کنید!

### ۲. محدودیت مدل:

- هر مدل یک ظرفیت محدود (مثلاً ۴k یا ۱۲۸k توکن) دارد. اگر چت خیلی طولانی شود، پیام‌های اولیه باید حذف شوند (First In, First Out) و گرنه ارور می‌دهد.

### ۳. کندی:

- پردازش متن‌های طولانی‌تر زمان بیشتری می‌برد.

راه حل حرفه‌ای (RAG) و (Vector DB) در سیستم‌های پیشرفته، به جای ارسال کل تاریخچه، فقط بخش‌های مرتبط را پیدا کرده و ارسال می‌کنیم (که موضوع جلسات آینده است).

## نوبت شماست!

کد بالا را در Google Colab کپی کنید و اجرا کنید. سعی کنید با مدل یک بازی ۲۰ سوالی انجام دهید تا ببینید چطور Context را حفظ می‌کند.

## فصل ۲۴: بهینه‌سازی توکن و مقدمه OpenRouter

### ۱. راهکارهای مدیریت حافظه و هزینه

مشکل افزایش تصاعدی توکن‌ها (که در فصل قبل دیدیم) چگونه حل می‌شود؟ مدرس با همفکری دانشجویان (متین و امیررضا) راهکارهای زیر را بررسی کرد:

#### الف) خلاصه‌سازی (Summarization)

- ایده: به جای ارسال ۱۰ صفحه چت قبلی، از یک مدل کوچک و ارزان (SLM) بخواهیم خلاصه مکالمه را بگیرد و فقط آن خلاصه را به مدل اصلی بفرستیم.
- مثال:
  - مکالمه ۱۰ دقیقه‌ای: بحث درباره مواد لازم قورمه‌سبزی، زمان پخت و ادویه‌ها.
  - خلاصه: "کاربر می‌خواهد قورمه‌سبزی بپزد و مواد و دستور پخت را گرفته است."

#### ب) قطعه‌بندی (Segmentation)

- تقسیم مکالمه به بخش‌های مختلف و فقط نگهداری بخش‌های مرتبط.

## ۲. معرفی سرویس OpenRouter

از اینجا به بعد، ابزار اصلی ما در دوره **OpenRouter.ai** خواهد بود.

### • چرا OpenRouter؟

- یک "دروازه" (Gateway) "واحد به تمام مدل‌های دنیاست.
- دسترسی همزمان به OpenAI, Claude (Anthropic), Gemini (Google), Llama (Meta) و صدها مدل دیگر.
- **مدل‌های رایگان**: دسترسی به مدل‌های قدرتمند رایگان (مثل سری Llama و Mistral) برای تمرین دانشجویان.
- نیازی به تغییر کد نیست (OpenAI Compatible) است.

### تکلیف دوم: گزارش State of AI

مدرس یک لینک بلاگ در سایت OpenRouter را به عنوان منبع مطالعاتی معرفی کرد.

<https://openrouter.ai/state-of-ai>

- **موضوع**: گزارش آماری دسامبر ۲۰۲۵ درباره وضعیت هوش مصنوعی.
- **اهمیت**: شناخت ترندهای بازار (کدام مدل بیشتر استفاده می‌شود؟ کدام کشورها پیشرو هستند؟).
- **نکته آماری**: ۴۷٪ مصرف توکن دنیا از آمریکا و ۹٪ از سنگاپور است.
- **وظیفه**: مطالعه گزارش و استخراج نکات کلیدی.

## ۳. دریافت کلید از OpenRouter

برای استفاده از این سرویس:

۱. به سایت [openrouter.ai](https://openrouter.ai) بروید.
۲. با گوگل لاگین کنید. (Sign up)
۳. در بخش **Keys** یک کلید جدید بسازید. (sk-or-...)
۴. در بخش **Credits**، اگر اعتبار اولیه ندارید، می‌توانید از مدل‌های بخش Free استفاده کنید.

تمرین عملی ۱۰ دقیقه‌ای

دانشجویان ۱۰ دقیقه فرصت دارند تا کد چت بات (فصل قبل) را اجرا کنند و با دستور `exit` آن را ببندند. پس از بازگشت، کد را برای اتصال به OpenRouter تغییر خواهیم داد.

## فصل ۲۵: تحلیل بازار AI و اتصال به OpenRouter

### ۱. تحلیل گزارش: OpenRouter پول کجاست؟

مدرس با بررسی نمودارهای گزارش، نکات طلایی زیر را استخراج کرد:

- **سلطه برنامه نویسی:** بخش بزرگی از مصرف توکن در دنیا مربوط به **Programming** است (بیشتر از مارکتینگ یا مالی).
- **پیام:** اگر برنامه نویس هستید و از AI برای کدنویسی (مثل Copilot یا Cursor) استفاده نمی کنید، در حال عقب ماندن از بازار هستید.
- **مدل های چینی:** مدل های مثل **DeepSeek** با قیمت بسیار پایین و کارایی بالا، حجم عظیمی از بازار را گرفته اند (به ویژه در چین و آسیا).
- **استارت آپ ها:** در بخش Apps، می توانید ببینید کدام اپلیکیشن ها از چه مدلی استفاده می کنند.
  - مثال: اپلیکیشن Chub.ai (یک سرویس شخصیت سازی) فقط در یک روز ۲۷۶ میلیارد توکن مصرف کرده است! این نشان دهنده پتانسیل عظیم بیزینس های مبتنی بر LLM است.

### ۲. پیدا کردن مدل های رایگان (Free Models)

در سایت OpenRouter، برخی مدل ها با برچسب **Free** مشخص شده اند. این مدل ها برای تست و یادگیری عالی هستند.

- **نحوه پیدا کردن:** به لیست Models بروید و فیلتر قیمت را روی Free بگذارید.
- **مثال ها:**
  - Mistral ۷B
  - Llama ۳ ۸B
  - Google Gemini Pro (گاهی رایگان)
  - حتی مدل های جدید مثل (Xiaomi که مدرس در کلاس کشف کرد!).

### ۳. اتصال پایتون به OpenRouter

مهم ترین بخش این فصل، تغییر کد OpenAI برای اتصال به OpenRouter است. چون OpenRouter از استاندارد OpenAI پیروی می کند، تغییرات بسیار جزئی است.

تغییرات لازم در کد:

۱. **Base URL:** آدرس پیش فرض (api.openai.com) باید به `https://openrouter.ai/api/v1` تغییر کند.

۲. **API Key:** کلید (sk-or-...) OpenRouter جایگزین کلید OpenAI شود.

۳. **Model Name:** نام مدل باید دقیقاً فرمت OpenRouter باشد (مثلاً `meta-llama/gpt-3.5-turbo` یا `llama/llama-3-8b-instruct`).

## کد نهایی اتصال به OpenRouter

```
from openai import OpenAI
```

۱. تنظیم کلاینت با آدرس و کلید OpenRouter

```
client = OpenAI(
 base_url="https://openrouter.ai/api/v1",
 api_key="sk-or-..." # کلید خود را اینجا بگذارید
)
```

۲. ارسال درخواست

```
response = client.chat.completions.create(
 # مدل رایگان (مثلاً Google Gemini ۲ Flash Experimental)
 model="google/gemini-۲,۰-flash-exp:free",
 messages=[
 {"role": "user", "content": "What is the meaning of life?"}
],
 # پارامترهای اختیاری خاص OpenRouter
 extra_body={
 "include_reasoning": True # اگر مدل قابلیت استدلال داشته باشد
 }
)
```

۳. دریافت پاسخ

```
print(response.choices[0].message.content)
```

**نکته کلیدی:** با همین کد بالا، شما می‌توانید به صدها مدل مختلف (Anthropic Claude, DeepSeek, Llama) وصل شوید، فقط کافیست نام `model` را تغییر دهید.

## ماموریت شما

در Google Colab، یک سلول جدید باز کنید. کد بالا را کپی کنید. یک مدل رایگان از سایت OpenRouter پیدا کنید (مثلاً `mistralai/mistral-۷b-instruct:free`) نام آن را در کد جایگزین کنید. سپس اجرا کنید و ببینید چطور بدون نیاز به VPN خاص یا پرداخت دلار، پاسخ می‌گیرید.

## فصل ۲۶: جایگزین‌های ایرانی و کار با AvalAI

### ۱. چالش مدل‌های رایگان و Rate Limit

در بخش قبل دیدیم که مدل‌های رایگان (Free) در OpenRouter گاهی اوقات پاسخ نمی‌دهند (خطای ۴۰۴ یا کندی شدید).

- **علت:** حجم بالای درخواست‌ها از سراسر دنیا.
- **محدودیت (Rate Limit):** اکانت‌های رایگان معمولاً محدودیت تعداد درخواست (مثلاً ۵۰ درخواست در روز یا ۱۰ درخواست در دقیقه) دارند.
- **راه حل:** یا شارژ دلاری OpenRouter (که از ایران سخت است) یا استفاده از سرویس‌های واسط ایرانی.

---

## ۲. معرفی سرویس AvalAI اول‌ای‌آی)

برای دانشجویانی که امکان پرداخت دلاری ندارند، سرویس‌های ایرانی مثل AvalAI گزینه مناسبی هستند.

- **مزیت:** پرداخت ریالی، سرعت مناسب (سرورهای داخل/نزدیک)، پشتیبانی از مدل‌های روز (GPT-۴o, Claude ۳.۵, Sonnet).
- **نحوه کار:**

۱. ثبت‌نام در سایت `avalai.ir`.
۲. شارژ حساب (مثلاً ۵۰ یا ۱۰۰ هزار تومان).
۳. دریافت API Key.

---

## ۳. اتصال به AvalAI با پایتون

از آنجا که AvalAI هم استاندارد OpenAI را رعایت می‌کند (OpenAI Compatible)، کدها تقریباً همان کدهای قبلی هستند، فقط `base_url` و `api_key` عوض می‌شود.

### الف) دریافت لیست مدل‌ها (با کتابخانه requests)

قبل از چت، ببینیم چه مدل‌هایی در دسترس است:

```
import requests

url = "https://api.avalai.ir/v1/models"
headers = {"Authorization": "Bearer YOUR_AVALAI_KEY"}

response = requests.get(url, headers=headers)
models = response.json()

چاپ نام اولین مدل و قیمت آن
first_model = models['data'][0]
print(f"Model: {first_model['id']}")
print(f"Input Price: {first_model['pricing']['input']}")
```

### ب) ارسال درخواست چت (با کتابخانه openai)

```

from openai import OpenAI

#تنظیم کلاینت برای AvalAI
client = OpenAI(
 base_url="https://api.avalai.ir/v1",
 api_key="aa-..." # کلید خود را از سایت اول بگیرید
)

response = client.chat.completions.create(
 model="gpt-4o-mini", # یا هر مدلی که در لیست دیدید
 messages=[
 {"role": "user", "content": "پایتون رو توی یه جمله تعریف کن"}
],
)

print(response.choices[0].message.content)

```

**نکته:** در AvalAI ممکن است نام مدل‌ها کمی متفاوت باشد (مثلاً پیشنهاد داشته باشد). حتماً لیست مدل‌ها را چک کنید یا از داکيومنت سایت استفاده کنید.

## ۴. مقایسه قیمت و کارایی

مدرس به بخش **Rankings** در OpenRouter برگشت تا نشان دهد کدام مدل‌ها برای چه کاری مناسب‌ترند:

- **کدنویسی:** مدل‌هایی مثل Sonnet ۳, ۵ و Claude و DeepSeek Coder.
- **ارزان و سریع:** GPT-4o-mini و Llama ۳.
- **تحلیل:** اگر پروژه سنگین دارید، هزینه را در نظر بگیرید. سرویس‌های ایرانی معمولاً کمی گران‌تر از قیمت اصلی دلاری هستند (به خاطر کارمزد واسطه)، اما دردسر پرداخت ارزی را حذف می‌کنند.

### هشدار امنیتی

هرگز API Key خود را در OpenRouter چه (AvalAI) یا در گروه‌های عمومی یا مخازن عمومی GitHub منتشر نکنید. اگر کلید لو برود، دیگران می‌توانند از اعتبار ریالی/دلاری شما استفاده کنند.

## فصل ۲۷: تبدیل متن به صدا (TTS) و معرفی ElevenLabs

### ۱. تبدیل متن به گفتار (Text-to-Speech) با AvalAI

مدرس نشان داد که چگونه می‌توان از API های AvalAI که به OpenAI متصل هستند (برای تولید فایل صوتی استفاده کرد).

## کد پایتون برای تولید صدا:

```
from openai import OpenAI
import warnings
warnings.filterwarnings("ignore") # حذف هشدارهای غیرمهم

اتصال به AvalAI
client = OpenAI(
 base_url="https://api.avalai.ir/v1",
 api_key="aa-..." # کلید AvalAI خود را اینجا بگذارید
)

مسیر فایل خروجی
speech_file_path = "speech.mp3"

ارسال درخواست ساخت صدا
response = client.audio.speech.create(
 model="tts-1", # مدل استاندارد OpenAI
 voice="alloy", # انتخاب گوینده (alloy, echo, fable, onyx, nova, shimmer)
 input="ما در حال ضبط دوره جدید عامل‌های هوش مصنوعی در تهران دیتا هستیم."
)

ذخیره فایل در دیسک
response.stream_to_file(speech_file_path)
print(f"فایل صوتی ذخیره شد {speech_file_path}")
```

- نکته درباره کیفیت فارسی: مدل‌های OpenAI مثل (tts-1 فارسی را پشتیبانی می‌کنند، اما ممکن است لهجه داشته باشند) گاهی شبیه لهجه افغانستانی یا غیربومی.

---

## ۲. چالش‌های TTS فارسی

- کمبود دیتا: مدل‌های متن‌باز (Open Source) با کیفیت بالا برای فارسی بسیار کم هستند.
- راهکار سازمانی: مدرس اشاره کرد که برای پروژه "کال سنتر" شرکت خودشان، مجبور شدند یک مدل اختصاصی را با دیتای تلفنی فارسی آموزش دهند (Fine-tune کنند)، چون مدل‌های عمومی کیفیت لازم را نداشتند.

---

## ۳. معرفی ElevenLabs (بهترین کیفیت صدا)

برای کسانی که به کیفیت صدای فوق‌العاده و طبیعی (بدون لهجه رباتیک) نیاز دارند، سرویس **ElevenLabs.io** معرفی شد.

- ویژگی‌ها:
  - **Voice Cloning:** می‌توانید صدای خودتان را آپلود کنید تا هوش مصنوعی با صدای شما صحبت کند.
  - پشتیبانی عالی از فارسی: مدل‌های جدید آن (Multilingual v2) فارسی را بسیار روان و با احساس بیان می‌کنند.



○ **Voice Changer:** تغییر صدای شما به صدای دیگر.

- **کاربرد:** تولید محتوا، کتاب صوتی، دوبله خودکار و دستیارهای صوتی پیشرفته.

مقایسه OpenAI TTS: ارزان تر و سریع تر است، اما ElevenLabs کیفیت و احساس بسیار بالاتری دارد.

## ۴. جمع بندی ساعت چهارم

در این ساعت کارهای زیر را انجام دادیم:

۱. عملیاتی شدن: کدنویسی پایتون در Colab را شروع کردیم.
۲. اتصال به سرویس ها: به OpenRouter و AvalAI وصل شدیم.
۳. مدیریت چت: سیستم Chat History (حافظه) را پیاده سازی کردیم.
۴. چندرسانه ای: اولین فایل صوتی را با کد تولید کردیم.

برنامه بعدی: بعد از استراحت، به سراغ ساخت **Web Application** با کتابخانه Gradio می رویم تا پروژه هایمان را گرافیکی کنیم.

### پایان ساعت چهارم

بخش کدنویسی پایه تمام شد .

بخش بعدی: ساخت رابط کاربری (UI) برای هوش مصنوعی

## فصل ۲۸: تولید تصویر (Image Gen) و مفهوم Base ۶۴

### ۱. تولید تصویر با DALL-E (کد پایتون)

مدرس ابتدا نحوه درخواست تصویر از OpenAI (یا AvalAI) را با استفاده از مدل dall-e-۳ نشان داد.

**کد پایتون:**

```
from openai import OpenAI
```

```
client = OpenAI(base_url="https://api.avalai.ir/v1", api_key="aa-VTe۰۸UK۹F۳۲DieBVswgAJSA۴Kkb۰AmdSMmf۲NJYAGesIzPbm")
```

```
response = client.images.generate(
```

```
 model="dall-e-۳",
```

```
 prompt="یک استاد مرد دانشگاهی در حال تدریس هوش مصنوعی در دانشگاه صنعتی",
```

```
 size="۱۰۲۴x۱۰۲۴",
```

```

quality="standard",
n=1,
)

image_url = response.data[0].url

```

- **تحلیل خروجی:** مدل‌های قدیمی‌تر مثل DALL-E ۳ ممکن است جزئیات فرهنگی (مثل پوشش در دانشگاه شریف) را دقیق رعایت نکنند، اما اصول کلی پرامپت را می‌فهمند.

## ۲. مفهوم کلیدی: Base64 چیست؟

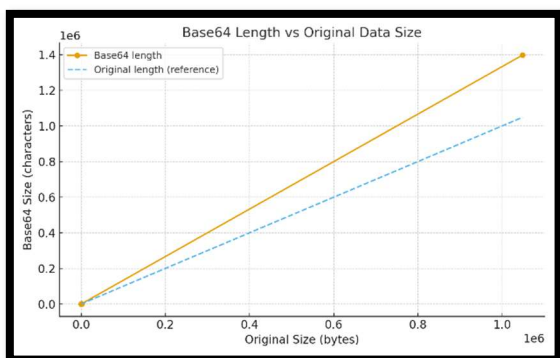
وقتی با API کار می‌کنیم، گاهی به جای URL تصویر، یک رشته متنی طولانی و عجیب دریافت می‌کنیم (یا باید ارسال کنیم). این همان Base64 است.

### تعریف ساده:

Base64 روشی است که فایل‌های باینری (عکس، صدا، ویدیو) را به یک رشته متنی امن (شامل حروف /, +, -, ۰-۹, A-Z, a-z) تبدیل می‌کند.

### چرا Base64 مهم است؟

۱. انتقال امن: بسیاری از سیستم‌ها (مثل JSON در API) فقط متن قبول می‌کنند Base64. عکس را به متن تبدیل



می‌کند تا بتواند از این کانال رد شود.

۲. افزایش حجم: فایل Base64 شده تقریباً ۳۳٪

حجم‌تر از فایل اصلی است (ضریب ۱.۳).

○ فرمول: هر ۳ بایت ورودی تبدیل به ۴ کاراکتر خروجی می‌شود.

○ طول رشته‌ی خروجی Base64 به راحتی

قابل محاسبه است، چون هر ۴ کاراکتر

Base64 به ۳ بایت ورودی تبدیل می‌شود.

○ فرمول کلی: اگر طول داده‌ی اصلی (تعداد

بایت‌ها) برابر n باشد:

$$\text{len}(\text{base64}) = 4 \times \lceil \frac{n}{3} \rceil$$

○

○ مثلاً اگر یک فایل ۱ مگابایتی (۱,۰۴۸,۵۷۶ بایت) رو Base64 کنید

$$\text{len}(\text{base64}) = 4 \times \lceil \frac{1048576}{3} \rceil = 1398108$$

۳. تبدیل دوطرفه :

○ **Encode:** تبدیل عکس به متن (برای ارسال به سرور).

○ **Decode:** تبدیل متن به عکس (برای نمایش به کاربر).

۴. به چه دردی می خورد؟

- وقتی سیستمی فقط می تونه متن بپذیره (نه باینری)، Base64 کمک می کنه.
- انتقال امن تر داده در JSON یا XML.
- استفاده در ایمیل ها، کوکی ها و ذخیره در پایگاه داده.

کد تبدیل Base64 به عکس (برای ذخیره خروجی):

```
from openai import OpenAI
import base64

if hasattr(response.data[0], "url") and response.data[0].url:
 print("Image URL:", response.data[0].url)
elif hasattr(response.data[0], "b64_json") and response.data[0].b64_json:
 image_base64 = response.data[0].b64_json
 image_bytes = base64.b64decode(image_base64)
 with open("output.png", "wb") as f:
 f.write(image_bytes)
 print("Image saved as output.png")
else:
 print("Unexpected response format:", response)
```

### ۳. استفاده از مدل های پیشرفته ( Replicate و Flux )

مدرس نشان داد که مدل های جدیدتر (مثل Flux یا سرویس Replicate) کیفیت بسیار بالاتری نسبت به DALL-E دارند و برای پروژه های تجاری (مثل تبلیغات محصول) مناسب ترند.

مثال عملی: کمپین تبلیغاتی ماگ استنلی

۱. ورودی :

- عکس محصول (یک ماگ Stanley).
- عکس کاراکتر (یا استایل خاص).
- پرامپت: "فضای شهری. (Urban Vibe)"

۲. فرآیند: عکس ها به صورت Base64 به مدل ارسال می شوند.

۳. خروجی: مدلی (مثل Flux) روی پلتفرمی مثل (Replicate)، ماگ را در دست یک مدل در خیابان قرار می دهد، طوری که نور و سایه ها کاملاً طبیعی باشند.

ایده بیزینسی: ساخت بات تلگرامی "مشاغل آینده" برای کودکان. والدین عکس کودک را می فرستند و بات عکس کودک را در لباس خلبان، دکتر یا فضانورد تحویل می دهد) با استفاده از همین تکنیک Face Swap یا Image-to-Image.

نکته فنی

هنگام ارسال عکس‌های با کیفیت بالا به API، حواستان به ساینز فایل باشد. اگر فایل خیلی بزرگ باشد، رشته Base64 آنقدر طولانی می‌شود که ممکن است درخواست تایم‌اوت شود یا هزینه توکن/ترافیک بالا برود. همیشه قبل از ارسال، تصاویر را تغییر ساینز (Resize) دهید.

## فصل ۲۹: بینایی و شنوایی ماشین و شروع کار با Gradio

### ۱. تبدیل صدا به متن (STT - Speech to Text)

مدرس نشان داد که چطور با استفاده از مدل (Whisper ساخت OpenAI، فایل صوتی ضبط شده توسط کاربر به متن تبدیل می‌شود).

کد پایتون) با استفاده از: AvalAI)

```
from openai import OpenAI # pip install -U openai

client = OpenAI(
 base_url="https://api.avalai.ir/v1",
 api_key="aa-RTcWXXHFGYHtEznCFEL9AVXr\۰۳W۲۰Jxn۳Wy\cUR۷wTpqnRX"
)

audio_file = open("/content/test.m4a", "rb")
transcription = client.audio.transcriptions.create(
 model="whisper-۱",
 file=audio_file,
 response_format="text"
)

print(transcription)
```

- تست عملی: مدرس صدای خود را ضبط کرد ("اندر احوالات شیخ ما...") و مدل آن را به دقت (حتی با ذکر نام گویندگان در مثال طنز) به متن تبدیل کرد.

### ۲. بینایی ماشین - (Vision) تحلیل عکس

قابلیت جذاب دیگر مدل‌های چندوجهی (Multimodal)، درک محتوای تصویر است. شما عکس را می‌دهید و مدل آن را توصیف می‌کند.

کد پایتون:

```
response = client.chat.completions.create(
 model="gpt-۴o", # مدل‌های ویژن‌دار
 messages=[
 {
 "role": "user",
 "content": [
 {"type": "text", "text": "این عکس چیه؟ به فارسی توضیح بده، "}]
]
)
```

```

 {
 "type": "image_url",
 "image_url": {
 "url": "https://example.com/soheil.jpg" #
 },
 },
],
)
print(response.choices[0].message.content)

```

یا آدرس عکس آپلود شده

- نتیجه: مدل عکس مدرس را دید و با دقت توصیف کرد "یک تیشرت قرمز رنگ به تن دارد، پس زمینه دیواری با رنگ روشن"...

### ۳. خداحافظی با محیط سیاه کدنویسی! شروع Gradio

تا اینجا همه کارها را در محیط متنی (Console) انجام دادیم. اما کاربر نهایی (User) نمی‌تواند با کد پایتون کار کند. او دکمه، کادر متن و منو می‌خواهد.

#### چرا Gradio؟

مدرس دو ابزار محبوب برای ساخت وب‌اپلیکیشن‌های دیتایی را مقایسه کرد:

۱. **Streamlit**: عالی برای داشبوردها و گزارش‌های داده‌محور (Data Apps).
۲. **Gradio**: (انتخاب این دوره) عالی برای دمو کردن مدل‌های هوش مصنوعی.
  - **مزیت اصلی**: مستقیماً داخل **Jupyter Notebook** یا **Colab** اجرا می‌شود و خروجی گرافیکی می‌دهد. نیاز به فایل جداگانه و سرور جدا ندارد.
  - **مالکیت**: متعلق به شرکت معروف **Hugging Face** است.

#### اولین برنامه "Gradio: سلام دنیا"

بیایید ساده‌ترین رابط کاربری ممکن را بسازیم: یک کادر ورودی نام، و یک کادر خروجی سلام.

```

import gradio as gr

۱. #تعریف تابع منطق برنامه
def greet(name):
 return "سلام " + name + " عزیز!"

۲. #ساخت رابط کاربری
demo = gr.Interface(
 fn=greet, # تابعی که باید اجرا شود
 inputs="text", # نوع ورودی (متن)
 outputs="text" # نوع خروجی (متن)
)

```

با اجرای این کد، یک لینک آبی رنگ ظاهر می‌شود (و یا مستقیماً در نوت‌بوک پنجره‌ای باز می‌شود) که کاربر می‌تواند نامش را بنویسد و دکمه Submit را بزند.

### تمرین سوم (تکمیل شد)

تا پایان این بخش، شما باید ۳ تمرین را انجام داده باشید :

۱. پاسخ به ۱۱۵ سوال مفهومی (از جلسات قبل).
۲. مطالعه و گزارش‌نویسی از مقاله آماری. OpenRouter.
۳. اجرای کدهای پارت ۳ و ۴ (تولید عکس و صدا) با استفاده از API Key خودتان.

## فصل ۳۰: اکوسیستم Hugging Face و توسعه رابط کاربری با Gradio

### ۱. پلتفرم Hugging Face قلب تپنده هوش مصنوعی)

مدرس توضیح داد که Hugging Face حکم "گیت‌هاب برای هوش مصنوعی" را دارد. این پلتفرم از سال ۲۰۱۸ و با معرفی معماری ترانسفورمرها (Transformers)، دنیای NLP را متحول و دموکراتیزه کرد.

#### بخش‌های مهم Hugging Face:

۱. **Models:** مخزن هزاران مدل متن‌باز (Open Source) مثل Llama, Mistral, BERT و مدل‌های جدید ویدیویی مثل Wan (۲۰۲۱).
۲. **Datasets:** مخزن داده‌های عظیم برای آموزش مدل‌ها.
۳. **Spaces:** جایی برای میزبانی دموها و وب‌اپلیکیشن‌های هوش مصنوعی که اکثرشان با Gradio ساخته شده‌اند.
  - مثال: شما می‌توانید سورس کد دموهای پیشرفته (مثل تولید انیمیشن) را در بخش Files هر Space ببینید و از آن‌ها یاد بگیرید.

**نکته Hugging Face:** شرکت Gradio را خرید تا بتواند ابزار ساخت دمو را مستقیماً در اکوسیستم خود ادغام کند.

### ۲. نصب و راه‌اندازی Gradio

برای استفاده از Gradio در گوگل کولب یا سیستم خودتان، ابتدا باید آن را نصب کنید:

!pip install gradio

## ساختار کلی یک برنامه: Gradio

یک برنامه Gradio همیشه سه بخش اصلی دارد:

۱. **Logic** (منطق): (تابعی که کار اصلی) محاسبه، پردازش AI و (...را انجام می دهد).
۲. **Interface** (رابط): (بخشی که ورودی ها و خروجی ها را تعریف می کند).
۳. **Launch** (اجرا): (دستور اجرای برنامه).

### مثال \Hello World :

```
import gradio as gr

۱. Logic
def greet(name):
 return "سلام" + name + "، خوش اومدی!"

۲. Interface
demo = gr.Interface(
 fn=greet, # تابع منطق
 inputs="text", # نوع ورودی: متن
 outputs="text" # نوع خروجی: متن
)

۳. Launch
demo.launch(share=True)
```

## ۳. ویژگی جادویی `share=True`:

مدرس یک قابلیت فوق العاده Gradio را معرفی کرد. وقتی در متد `launch` مقدار `share=True` را قرار می دهید:

- یک لینک عمومی (Public URL) موقت (معتبر برای ۷۲ ساعت) تولید می شود.
- شما می توانید این لینک را برای مشتری، استاد یا دوستانتان بفرستید.
- آن ها می توانند بدون نصب هیچ چیزی، با مدل هوش مصنوعی شما که روی سیستم شما (یا کولب) اجرا شده، کار کنند.

**کاربرد بیزینسی:** فرض کنید می خواهید یک پروژه را به کارفرما تحویل دهید. به جای اینکه بگویید "پایتون نصب کن"، لینک Gradio را می فرستید تا دمو را ببیند و تایید کند.

## ۴. کامپوننت های متنوع Gradio

Gradio فقط متن نیست. مدرس به تنوع ورودی/خروجی ها اشاره کرد:

- **تصویر (Image):** برای پروژه‌های پردازش تصویر.
- **صدا (Audio):** برای ضبط صدا (میکروفون) یا آپلود فایل.
- **نقشه (Map):** نمایش داده‌های مکانی (مثل قیمت خانه در مناطق مختلف روی نقشه. Plotly)
- **اسلایدر و دکمه:** برای تنظیم پارامترها.

در ادامه، قرار است این کامپوننت‌ها را به توابع هوش مصنوعی (که در ساعت قبل نوشتیم) متصل کنیم تا یک اپلیکیشن کامل بسازیم.

## نکته آموزشی

در Hugging Face Spaces، می‌توانید روی دکمه **Files** هر پروژه کلیک کنید و فایل `app.py` آن را ببینید. این بهترین راه برای یادگیری ترفندهای پیشرفته Gradio از کدنویسان حرفه‌ای دنیاست.

# فصل ۳۱: هوش مصنوعی و طراحی وب: از رویاپردازی تا واقعیت با Gradio

## ۱. لحظه‌ای برای شگفتی: ابزار Lovable.dev

یکی از دانشجویان ابزاری به نام **Lovable** یا مشابه آن مثل **V0.dev** را معرفی کرد که با یک پرامپت متنی، کل کد Frontend (رابط کاربری) سایت را با **React** و **Tailwind** می‌زند و حتی **Deploy** می‌کند!

- **واکنش مدرس:** "افسردگی گرفتم! چرا باید دولوپر استخدام کرد؟"
- **نتیجه‌گیری مهم:** این ابزارها برای ساخت محصول نهایی (**Production**) یا صفحات لندینگ عالی هستند. اما **Gradio** لیگ متفاوتی دارد.
- **جایگاه Gradio:** گریدیم برای **POC** (اثبات مفهوم) و **MVP** (حداقل محصول پذیرفتنی) داخلی استفاده می‌شود. وقتی می‌خواهید مدل هوش مصنوعی خود را تست کنید یا به هم‌تیمی‌هایتان نشان دهید، **Gradio** سریع‌ترین راه است (بدون درگیر شدن با **CSS** و **React**).

## ۲. پروژه عملی: ساخت اپلیکیشن متن به صدا (TTS App)

بیا ببینیم آن کد پایتونی که صدا تولید می‌کرد را به یک وب‌اپلیکیشن تبدیل کنیم تا کاربر بتواند متن بنویسد و فایل صوتی دانلود کند.

### منطق برنامه (Function)

ابتدا تابعی می‌نویسیم که متن را بگیرد و فایل صوتی برگرداند:

```
from openai import OpenAI
```



```

import gradio as gr

#تنظیم کلاینت AvalAI
client = OpenAI(
 base_url="https://api.avalai.ir/v1",
 api_key="aa-..."
)

def text_to_speech(text):
 try:
 # ۱. درخواست ساخت صدا
 response = client.audio.speech.create(
 model="tts-1",
 voice="alloy",
 input=text
)

 # ۲. ذخیره فایل
 output_path = "output_audio.mp3"
 response.stream_to_file(output_path)

 # ۳. بازگرداندن مسیر فایل (برای Gradio)
 return output_path

 except Exception as e:
 # در صورت خطا، متن خطا را برگردان
 return str(e)

```

## رابط کاربری (Interface)

حالا این تابع را به کامپوننت‌های گرافیکی وصل می‌کنیم:

```

demo = gr.Interface(
 fn=text_to_speech, # تابع منطق
 # ورودی: یک کادر متنی بزرگ
 inputs=gr.Textbox(
 lines=۳,
 placeholder="متن خود را اینجا بنویسید", ...
 label="متن ورودی"
),
 # خروجی: پخش‌کننده صدا
 outputs=gr.Audio(
 label="فایل صوتی خروجی"
),
 title="تبدیل متن به صدا با هوش مصنوعی",
 description="متن فارسی یا انگلیسی وارد کنید و دکمه Submit را بزنید."
)

demo.launch(share=True)

```

## تحلیل کد:

۱. `gr.Textbox` به جای `inputs="text"` کلاس `Textbox` استفاده کردیم تا بتوانیم تعداد خطوط (`lines=۳`) و متن راهنما (`placeholder`) را تعیین کنیم.
۲. `gr.Audio` خروجی را به صورت یک پلیئر فایل صوتی نمایش می‌دهد که دکمه پخش و دانلود دارد.
۳. `try-except` حتماً بلوک مدیریت خطا بگذارید تا اگر اینترنت قطع شد یا اعتبار تمام شد، برنامه کرش نکند و پیام خطا نمایش دهد.

## ۳. تمرین شما

این کد را در گوگل کولب اجرا کنید.

۱. متن "سلام، به دوره هوش مصنوعی خوش آمدید" را تایپ کنید.
۲. دکمه `Submit` را بزنید.
۳. صبر کنید تا فایل صوتی تولید شود و آن را پخش کنید.
۴. لینک عمومی (`gradio.live`) را برای دوستان بفرستید تا او هم تست کند.

### ماموریت بعدی

در بخش بعد، می‌خواهیم این فرآیند را برعکس کنیم: صدا بگیریم و متن تحویل دهیم (STT) و حتی یک چت‌بات کامل بسازیم. آماده باشید!

## فصل ۳۲: چشم‌های بینا: ساخت اپلیکیشن تحلیل تصویر و داشبورد با Gradio

### ۱. چالش ۶۴ و Vision API

برای ساخت اپلیکیشنی که کاربر در آن عکس آپلود کند و هوش مصنوعی آن را تحلیل کند، یک چالش فنی کوچک داریم:

- API‌های (OpenAI و AvalAI معمولاً آدرس URL عکس را می‌پذیرند).
- اما در Gradio، وقتی کاربر عکس را از کامپیوترش آپلود می‌کند (یا با وب‌کم می‌گیرد)، ما فایل محلی (Local File) داریم، نه URL اینترنتی.
- راه حل: تبدیل فایل عکس به فرمت `Base64` و ارسال آن درون متن درخواست (Payload).

تابع کمکی برای تبدیل عکس به: `Base64`

```
import base64
```

```
def encode_image(image_path):
 with open(image_path, "rb") as image_file:
 return base64.b64encode(image_file.read()).decode('utf-8')
```

## ۲. کد اپلیکیشن تحلیل تصویر (Vision App)

این برنامه یک عکس از کاربر می‌گیرد، آن را به Base64 تبدیل می‌کند، به GPT-4o می‌فرستد و تحلیل فارسی دریافت می‌کند.

```
import gradio as gr
from openai import OpenAI

client = OpenAI(
 base_url="https://api.avalai.ir/v1",
 api_key="aa-..."
)

def analyze_image(image_path):
 # ۱. تبدیل عکس به Base64
 base64_image = encode_image(image_path)

 # ۲. ارسال به مدل
 response = client.chat.completions.create(
 model="gpt-4o",
 messages=[
 {
 "role": "user",
 "content": [
 {"type": "text", "text": "این تصویر را به زبان فارسی و با جزئیات کامل تحلیل کن، { }."},
 {"type": "image_url",
 "image_url": {
 "url": f"data:image/jpeg;base64,{base64_image}"
 }
]
 }
]
)
 return response.choices[0].message.content

۳. رابط کاربری Gradio
demo = gr.Interface(
 fn=analyze_image,
 inputs=gr.Image(type="filepath", label="تصویر را آپلود کنید یا با وب‌کم بگیرید،"),
 outputs=gr.Textbox(label="تحلیل هوش مصنوعی", lines=5),
 title="تحلیل تصویر با هوش مصنوعی، (GPT-4o)",
 description="یک عکس بارگذاری کنید تا هوش مصنوعی جزئیات آن را شرح دهد."
)

demo.launch(share=True)
```

**تست‌های انجام شده در کلاس:**

۱. سلفی مدرس با وب کم: مدل با دقت تشخیص داد "فردی با لباس سبز، هدست روی سر، دست خمیده که هدست را نگه داشته، پس زمینه کاغذ دیواری"...
۲. تحلیل داشبورد PowerBI: مدرس عکس یک داشبورد مدیریتی مدارس را آپلود کرد.

### ۳. شاهکار هوش مصنوعی: تحلیل داده از روی عکس

وقتی عکس داشبورد PowerBI (شامل نمودارها و اعداد ریز) به مدل داده شد، نتیجه حیرت‌انگیز بود:

- تشخیص اعداد: "تعداد دانش‌آموزان پسر: ۱۳۱۵ نفر، دختر: ...، کل معلم‌ها: ۳۱۴ نفر" دقیقاً همان اعدادی که در عکس ریز نوشته شده بود.
  - تحلیل نمودار: "نمودار سمت چپ توزیع دانش‌آموزان را نشان می‌دهد... نمودار پایین درصد مشارکت مدارس مختلف مثل مدرسه آرنا را نشان می‌دهد".
  - نتیجه‌گیری مدیریتی: مدل توانست وضعیت کلی (مثلاً "ارزیابی بسیار ضعیف") را از روی رنگ‌بندی و اعداد تشخیص دهد.
- کاربرد بیزینسی: شما می‌توانید اپلیکیشنی بسازید که مدیران شرکت‌ها عکس گزارش‌های ماهانه را آپلود کنند و "خلاصه مدیریتی" و "نقاط بحرانی" را تحویل بگیرند، بدون اینکه نیاز باشد به دیتابیس وصل شوند!

### ۴. نکته درباره دیپلوی (Deployment)

دانشجویان پرسیدند چگونه این اپلیکیشن را دائمی کنیم؟ چون لینک Gradio بعد از ۷۲ ساعت منقضی می‌شود.

- **Hugging Face Spaces:** بهترین و رایگان‌ترین راه. کد را آنجا آپلود می‌کنید و همیشه آنلاین می‌ماند.
- **سرور شخصی:** استفاده از Nginx و Gunicorn روی یک VPS لینوکسی.
- **VS Code:** اگر کد را در VS Code سیستم خودتان اجرا کنید و `share=True` تنظیم کنید، روی `localhost:۷۸۶۰` همیشه برای خودتان در دسترس است.

#### هشدار مصرف

ارسال عکس به API (مخصوصاً با کیفیت بالا) هزینه توکن بیشتری نسبت به متن دارد. هنگام تست، از عکس‌های کم‌حجم استفاده کنید یا سایز عکس را قبل از ارسال کاهش دهید.

# فصل ۳۳: ساخت چت بات تعاملی با Gradio و جمع بندی جلسه

## ۱. ساخت چت بات با حافظه (Stateful Chatbot)

تا الان، توابعی که نوشتیم "یک بار مصرف" بودند (یک سوال، یک جواب). اما چت بات واقعی باید تاریخچه مکالمه (History) را بداند تا وقتی می گوییم "پسته چطور؟"، بفهمد منظورمان "پسته برای شب یلدا" است.

استفاده از `gr.ChatInterface`

Gradio یک کامپوننت مخصوص و آماده به نام `ChatInterface` دارد که مدیریت تاریخچه و ظاهر چت را خودش انجام می دهد.

کد نهایی چت بات:

```
import gradio as gr
from openai import OpenAI

client = OpenAI(
 base_url="https://api.avalai.ir/v1",
 api_key="aa-..."
)

تابع منطق چت
لیست پیام های قبلی (history (پیام جدید) و message: این تابع باید دو ورودی بگیرد
def chat_logic(message, history):

 # OpenAI ساختن لیست پیام ها برای ارسال به
 messages_payload = [
 {"role": "system", "content": "تو یک دستیار بامزه و پرانرژی هستی که به زبان فارسی و با لحن شوخ طبع صحبت می کنی"}
]

 # اضافه کردن تاریخچه مکالمات قبلی به لیست
 for user_msg, bot_msg in history:
 messages_payload.append({"role": "user", "content": user_msg})
 messages_payload.append({"role": "assistant", "content": bot_msg})

 # اضافه کردن پیام جدید کاربر
 messages_payload.append({"role": "user", "content": message})

 # دریافت پاسخ از مدل
 response = client.chat.completions.create(
 model="gpt-4o",
 messages=messages_payload
)

 return response.choices[0].message.content
```

```
رابط کاربری مخصوص چت ۲.
demo = gr.ChatInterface(
 fn=chat_logic,
 title="چت بات شوخ طبع تهرانی",
 description="! هر سوالی داری بپرس، ولی منتظر جوابهای بامزه باش"
)

demo.launch(share=True)
```

## تست عملی در کلاس:

- کاربر: "می خوام برای دخترام لباس شب یلدا بخرم".
- بات: (با لحن طنز) "چه کار هیجان انگیزی! مواظب باش طرح هندونه نگیری که حسرت دونه هاش بمونه..."
- کاربر: "کیک چطور؟"
- بات: (درک "Context" آخ آخ کیک عالیه! فکر کن کیک شکل هندونه باشه) "..."
- کاربر: "پسته چطور؟"
- بات: "پسته که دیگه گنج قارونه! اگه وسعت می رسه بخر..."

تحلیل: بات توانست بدون اینکه دوباره کلمه "یلدا" را بشنود، ارتباط بین کیک، پسته و یلدا را حفظ کند. این قدرت مدیریت history است.

## ۲. جمع بندی جامع جلسه امروز

امروز یک سفر طولانی و پربار را از مفاهیم پایه تا ساخت محصول طی کردیم:

۱. مفاهیم زیرساختی: توکن چیست؟ LLM چطور کار می کند؟ API و JSON چه اهمیتی دارند؟
۲. ابزارهای اتصال: یاد گرفتیم چطور با curl و پایتون به API وصل شویم (OpenRouter و AvalAI).
۳. مدل های چندوجهی: کار با مدل های تولید تصویر (DALL-E/Flux)، صدا (TTS)، و بینایی ماشین (Vision).
۴. ساخت محصول (Web App): با استفاده از Gradio، کدهای خشک پایتون را به اپلیکیشن های تعاملی تبدیل کردیم که می توانیم به دیگران ارائه دهیم.

## ۳. تمرین های هفته آینده

برای تسلط بر این مطالب، سه ماموریت دارید:

۱. مرور مفاهیم: مطالعه ۱۱۵ سوال و جوابی که فایلش ارسال شد.
۲. گزارش خوانی: مطالعه و بررسی آماری فایل OpenRouter نیاز به ارسال نیست، فقط برای بحث در کلاس.
۳. پروژه عملی (مهم ترین بخش):
  - اجرای تمام کدهای پارت ۳، ۴ و ۵ در گوگل کولب خودتان.

- ساخت یک چت بات یا ابزار تحلیل تصویر اختصاصی با سلیقه خودتان و آپلود کد آن در گیت هاب.

## پایان جلسه پنجم

شما اکنون می‌توانید یک ایده هوش مصنوعی را از ذهن به یک نمونه اولیه (Prototype) قابل استفاده تبدیل کنید.

خسته نباشید!